How to Break It Down for Building It Up? Theory-Guided Graph Decomposition Learning for Spatiotemporal Traffic Prediction

Jiahao Ji, Jingyuan Wang, Yu Mou, Cheng Long, and Junjie Wu

***Abstract*—Traffic state prediction based on spatiotemporal data has become a prominent focus in data-driven AI research. While significant progress has been made, most mainstream approaches assume *uniform* spatial and temporal correlations across conditions and use shared parameters for all scenarios. This simplification overlooks the complexity and heterogeneity inherent in human mobility patterns, often leading to suboptimal predictions. Recently, methods adopting the *"decompose, then predict" (DTP)* paradigm have gained traction. These methods break down data into smaller, manageable subcomponents, each predicted using dedicated parameters. Although effective in practice, DTP methods face unresolved theoretical questions: What type of decomposition truly makes subcomponents more manageable than the original data? To address this, we present an information theory-based analysis that derives sufficient conditions for a decomposition algorithm to reduce data-induced prediction errors. These conditions suggest that an effective algorithm should ensure decomposed components are as independent as possible, a principle we term the *Component Independence Principle*. Guided by this principle, we introduce the Theory-guided Graph Decomposition Learning (TGDL) framework, which decomposes graph-based multivariate time series data into approximately independent subgraph components that are easier to predict than the original data. Moreover, TGDL is a portable framework that can be integrated into any graph-based traffic prediction model to improve its predictive performance. Extensive experiments on four public datasets demonstrate the effectiveness of our approach. With a solid theoretical foundation, our TGDL enhances the performance of diverse traffic prediction models, yielding an average improvement of 19.37% across experiments.**

***Index Terms*—Decomposition prediction, Fano's inequality, data-induced error, theory-guided model design, time series prediction.**

Received 17 November 2024; revised 18 December 2025; accepted 30 December 2025. Date of publication 12 January 2026; date of current version 3 April 2026. The work of Jingyuan Wang was supported by the National Natural Science Foundation of China under Grant 72222022 and Grant 72171013. The work of Junjie Wu was supported in part by the National Natural Science Foundation of China under Grant 72242101, Grant 72031001, and Grant 72595861, and in part by the Outstanding Young Scientist Program of Beijing Universities under Grant JWZQ20240201002. Recommended for acceptance by R. Cucchiara. *(Corresponding author: Jingyuan Wang.)*

Jiahao Ji and Yu Mou are with the School of Computer Science and Engineering, Beihang University, Beijing 100191, China, and also with the MOE Engineering Research Center of Advanced Computer Application Technology, Beihang University, Beijing 100191, China.

Jingyuan Wang is with the School of Economics and Management, Beihang University, Beijing 100191, China, and also with the School of Computer Science and Engineering, Beihang University, Beijing 100191, China (e-mail: jywang@buaa.edu.cn).

Cheng Long is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798.

Junjie Wu is with the School of Economics and Management, Beihang University, Beijing 100191, China, and also with the MIIT Key Laboratory of Data and Decision Intelligence, Beihang University, Beijing 100191, China.

Our code is available at https://github.com/bigscity/TGDL.

Digital Object Identifier 10.1109/TPAMI.2026.3651246

I. Introduction

In recent years, the widespread adoption of the Internet of Things (IoT) and GPS technologies has enabled large-scale spatiotemporal data collection [1], [2]. This progress has fueled significant advancements in spatiotemporal AI technologies [3], [4], [5], [6]. Among these, traffic state prediction remains a critical and enduring area of research [7], [8], [9]. High-performance traffic prediction models are crucial for various real-world transportation and urban management applications [10], [11]. For example, accurate traffic speed prediction models can enhance online map and navigation services by improving traffic information [12], [13], while precise traffic flow prediction models can help urban transportation departments manage traffic congestion more effectively [14], [15].

A typical traffic prediction model leverages historical data, such as speed or flow, to forecast future conditions for specific geographic units (e.g., road segments or land parcels). Early studies in the literature treated traffic states as conventional time series and employed shallow models for prediction, such as ARIMA [16], Kalman filters [17], and similar approaches. As deep learning technologies continue to advance, neural network-based models have become the dominant approach for traffic prediction [7]. These deep models build on classical neural network architectures as their backbone, extending them to effectively capture the temporal and spatial correlations in traffic state data. For temporal correlation modeling, sequential deep learning models—such as Recurrent Neural Networks (RNN) [18], [19], Temporal Convolutional Networks (TCN) [20], [21], and Transformers [22], [23], [24]—are widely adopted. For spatial correlation modeling, Convolutional Neural Networks (CNN) [23], [25] and Graph Neural Networks (GNN) [15], [26], [27] are commonly used. Despite their success, most deep learning-based models assume uniform spatial and temporal correlations across

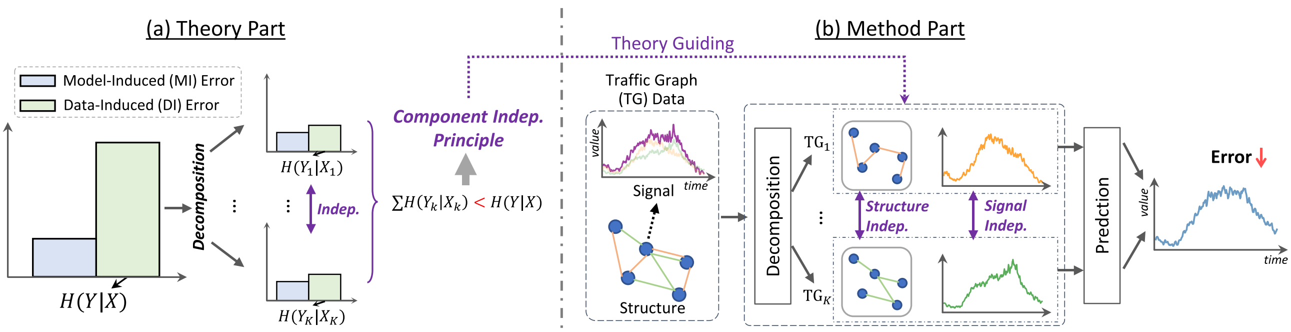


Fig. 1. Illustration of our theoretical and methodological innovations. Indep: Independence.

all geographic units. Under this assumption, a single set of parameters is learned from the data of all units and applied universally to predict traffic conditions across different locations and time periods. However, real-world traffic data is inherently complex and heterogeneous [15], [28], [29]. Human mobility patterns—shaped by commuting, entertainment, and business activities—introduce varying spatial and temporal correlations (see Sect. I.B of the Supplementary Materials for illustration). Many studies have demonstrated that traffic data can be decomposed into multiple subcomponents, each exhibiting relatively simple yet distinct spatial and temporal patterns [30], [31], [32].

To address these complexities, a more effective modeling paradigm, known as the decompose-then-predict (DTP) approach, has gained attention in traffic prediction. In this approach, traffic data is decomposed into distinct components, each modeled and predicted with separate parameters. Early studies employed traditional signal decomposition tools, such as seasonal-trend decomposition [33] and wavelet [34], [35], to preprocess traffic data into multiple components before applying models for prediction. More recent research integrates decomposition techniques directly into deep neural networks, facilitating end-to-end multi-component traffic prediction models [20], [36], [37], [38]. Empirical evidence from existing studies indicates that the decomposition modules within DTP frameworks significantly enhance prediction performance [32], [39], [40]. Although many studies have emphasized the potential of DTP methods, the following fundamental questions about these decomposition methods remain unanswered.

Q1. What type of decomposition can theoretically ensure that the decomposed components exhibit higher predictability than the original spatiotemporal data? DTP paradigm adopts a "divide-and-conquer" strategy to deconstruct a complex problem into simpler, more manageable sub-problems. However, not all decomposition methods lead to improved predictive performance. For example, if a traffic series is split into N components, each representing $1/N$ of the original series, the prediction complexity of the individual components may remain unchanged. Therefore, rigorous theoretical analysis is crucial to identify decompositions that can effectively reduce prediction complexity.

Q2. Can we design a decomposition-based predictive framework with theoretical guidance to enhance the performance of various traffic prediction models? If a theoretical solution to Q1 is obtained, the next step is to develop a predictive modeling framework based on decomposition theory. This predictive framework should ensure that the decomposed components are more predictive than the original data. Furthermore, to function as a true "framework," it must be adaptable to various traffic prediction models, maintaining compatibility across different modeling approaches.

To address these two questions, our study consists of two parts: a *theory part* and a *method part*, as illustrated in Fig. 1. **In the theory part**, we address Q1 through a theoretical analysis grounded in Information Theory. The core of Q1 concerns how to quantify and enhance the intrinsic predictability of data. Information theory provides a rigorous framework for this purpose, with entropy serving as a principled metric to measure uncertainty—lower entropy corresponds to higher predictability [41], [42]. Accordingly, a decomposition strategy that reduces entropy is expected to improve predictability, and we leverage this rationale to derive a sufficient condition for effective decomposition. In the context of traffic prediction, the goal is to forecast future states Y from historical data X. The task difficulty, depending on the uncertainty in predicting Y given X, is quantified by the conditional entropy $\mathrm{H}(Y|X)$. If X and Y are decomposed into subcomponents X_k and Y_k, and the total conditional entropy across all subproblem, $\mathrm{H}(Y_k|X_k)$, is lower than $\mathrm{H}(Y|X)$, the subproblems become inherently easier to predict. Intuitively, if these subcomponents are mutually independent, the predictive uncertainty for each subproblem becomes more manageable, resulting in a reduction in overall uncertainty. We formalize this as the Component Independence (CI) Principle, which serves as a theoretical foundation for guiding decomposition-based learning. Formally, we model traffic prediction as a *noisy channel transmission and decoding process*, where historical states are treated as messages transmitted through a noisy channel and decoded into future states by a prediction function. Within this framework, we identify two main sources of information loss: i) transmission noise in the channel (*data-induced errors*), and ii) decoding errors in the prediction process (*model-induced errors*). Building on this model and leveraging *Fano's Inequality* [43], we show that, if the decomposed components are mutually independent, the total data-induced error for all subproblems is lower than that of the undecomposed problem. Consequently, the decomposed subproblems exhibit higher predictability than the original one.

In the method part, guided by the above theoretical analysis of Component Independence (CI) Principle, we design a novel Theory-guided Graph Decomposition Learning (TGDL) framework to address Q2. In TGDL, geographical units and their traffic states are represented as a graph with dynamic signals. The graph is decomposed into several independent subgraphs through two key modules: *structure decomposition* and *signal decomposition*. The structure decomposition employs a soft graph mask matrix along with structure independence regularization to partition the original graph into approximately orthogonal subgraph structures. The signal decomposition uses a residual decomposition flow to disentangle graph signals into multiple subgraph signal components. To ensure the independence of these components, a signal independence regularization is designed to minimize their mutual information. By generating orthogonal subgraph structures with independent signals, TGDL adheres to the CI Principle. Each subgraph is assigned a specialized prediction model, and the predictions from all subgraphs are aggregated to produce the final traffic forecast. As a general framework, TGDL is compatible with and boosts the performance of any graph-based traffic prediction model that can serve as a subgraph predictor. We demonstrate the effectiveness of our theory and framework through extensive experiments on four public traffic datasets.

The contributions of our study are summarized as:

- To our best knowledge, this study provides the first theoretical analysis for identifying decomposition algorithms that enhance traffic prediction. It establishes a theoretical foundation for the decompose-then-predict paradigm.
- We propose a novel TGDL framework, guided by our theoretical analysis, which decomposes graph-based traffic data into approximately independent subgraph components. These components are easier to predict than the original data, and the framework is designed to improve the performance of any graph-based traffic prediction model with an encoder-decoder architecture.
- Extensive experiments on four real-world traffic datasets demonstrate the performance gains achieved by integrating the TGDL plugin into various graph-based models. Results show that TGDL improves the performance of a wide range of traffic models by an average of 19.37%.

The remainder of the paper is organized as follows. Section II provides the related work. Section III presents the theoretical analysis addressing Q1 in the Introduction part, while Section IV introduces the design of our TGDL framework for Q2, guided by the theoretical insights. Section V details the experimental evaluation. The conclusion is provided in Section VI.

II. Related Work

Traffic Prediction is a classic problem in spatiotemporal series prediction. Early approaches framed traffic prediction as a regular time series forecasting problem, using shallow models like ARIMA [16] and Kalman filtering [44]. Recently, neural network-based deep learning models have become mainstream, which extend classic deep learning model architectures to capture the spatiotemporal correlations in traffic data [5], [28]. For temporal correlation modeling, sequential deep learning models—such as Recurrent Neural Networks (RNN) [18], [29], Long Short-Term Memory (LSTM) [45], Temporal Convolutional Networks (TCN) [20], [21], and Transformers [22], [23], [24]—serve as popular backbone models. Variants of these models enhance temporal feature extraction by incorporating periodicity [45], [46], rhythm [11], [31], congestion propagation [14], and other traffic-specific factors. For spatial correlation modeling, the backbone models often include Convolutional Neural Networks (CNN) [23], [25] or Graph Neural Networks (GNN) [15], [26], [27]. In these approaches, urban geographic units are represented as cells within a raster grid [47] or as vertices within a graph [10]. Models integrate topological information such as distances [19], connection relations [15], [47], spatial semantic context [19], [24], and human mobility behaviors [20], [48] to capture complex spatial relationships between cells and vertices. Recently, some models based on pure multi-layer perceptrons (MLP) have attracted attention due to their efficiency and scalability [27], [49]. In addition, there are also some studies that try to introduce LLM or its ideas into the field of traffic prediction [23], [48], [50]. From a training perspective, traffic prediction models fall into the *local paradigm* and *global paradigm* [51]. The local paradigm uses data from specific road segments or land parcels [52], a common approach for simpler, shallow models. The global paradigm, primarily adopted by deep learning models, treats all geographical units collectively, drawing on data from all units for a richer training dataset and enhanced correlation modeling [24], [27], [47]. While the global paradigm has greatly advanced traffic prediction, it often overlooks the variability among different traffic data components, which is our focus.

Decomposition Prediction aims to decompose a signal sequence into components for separate prediction, each influenced by distinct latent factors. Originating from time series forecasting [53], [54], it provides valuable insights and aids decision-making in finance [55], health [56], [57], meteorology [58] and etc. Popular methods include seasonal-trend decomposition [33], wavelet decomposition [34], and empirical mode decomposition [59]. Recently applied to traffic data, decomposition prediction follows two paradigms: two-stage and end-to-end. The two-stage paradigm preprocesses traffic data into components, then uses deep learning to model each [60], [61]. The end-to-end paradigm incorporates decomposition within the prediction model, handling multiple traffic data components simultaneously [37], [38]. Related methods often decompose traffic signals into diffusion and non-diffusion [36], high and low frequency [32], [39], trend and seasonality [40], and normal and abnormal signals [62]. These decomposed prediction methods have shown empirical improvements by decomposing complex signals into simple ones. However, most of them rely on heuristic or domain-specific decomposition strategies, without formal guarantees of their effectiveness. Besides, they are often designed in an ad hoc manner, tightly coupled to specific tasks or architectures, limiting their generalizability and portability. In contrast, this paper explores an information-theoretic perspective to identify effective decomposition and designs a general and pluggable framework that can be integrated into various backbone models. More details can be found in Sect. III.A of the Supplementary Materials.

Graph Decomposition aims to break down complex graphs into simpler subgraphs [63], [64]. Originally developed for graph clustering, it seeks to create balanced partitions with minimal inter-partition edges, also known as cuts [65]. Given its combinatorial complexity, various approximation algorithms have been developed, including multilevel partitioning [66], spectral clustering [67], and simulated annealing [68]. Recent approaches integrate deep learning techniques like contrastive learning [69] and adversarial learning [70]. Graph decomposition is also applied in distributed graph computing for large-scale graphs [71], though this lies beyond our study's scope. In summary, conventional graph decomposition methods primarily focus on the graph structure itself, aiming to uncover meaningful structural patterns. In contrast, this work differs in both the object and the goal of decomposition: it targets spatiotemporal data that integrate graph structures and series signals, and it pursues decomposition not merely for pattern extraction, but explicitly to enhance predictive performance.

Theory-guided Deep Model Design integrates scientific theory and domain knowledge into deep learning models to ensure physical plausibility and robust generalizability [72]. This approach has shown benefits across fields like transportation [11], meteorology [3], and fluid mechanics [73]. Some studies also leverage cross-domain theories, such as predictability theory from signal processing for human mobility modeling [41], potential field theory from classical mechanics for flood mapping [74] and traffic flow modeling [75], and causal inference theory from econometrics for spatiotemporal graph prediction [47], [76], [77]. In this paper, we apply error decomposition [78] and predictability theories [79] to guide traffic prediction model design. Error decomposition theory classifies prediction errors into bias, variance, and noise, with bias and variance being model-induced and noise data-induced [80]. Most research focuses on reducing model-induced errors, often overlooking data-induced errors. To address this, we apply predictability theory (using Fano's inequality) not just to assess predictability [79] but to enhance it by decomposing data into mutually independent components.

III. The Theory Analysis

In this section, we present an information theory-based analysis of prediction errors in the decomposition prediction problem. From this foundation, we derive a sufficient condition under which the prediction errors for the decomposition problem are lower than those of the original problem.

A. Information Theory Model for Traffic Prediction

Let X denote historical traffic states and Y the future traffic states. Here, X and Y are random variables. The traffic state prediction problem involves constructing a function:

$$\hat{Y} = f(X, \theta), \tag{1}$$

where $\hat{Y}$ represents the predicted result and θ the parameters of the prediction function. The goal is to optimize θ to minimize prediction errors across all training samples.

TABLE I
Analogy Between the Noisy Channel Model and Traffic Prediction

Notation	Noisy Channel Model	Traffic Prediction
$\hat{Y} = f(X, \theta)$	Decoder	Prediction Model
Y	Source Message	Future Traffic State
$\tilde{X}$	Compressed Message	—
X	Compressed Message with Noise	Historical Traffic State
$\hat{Y}$	Reconstructed Message	Predicted Future Traffic State

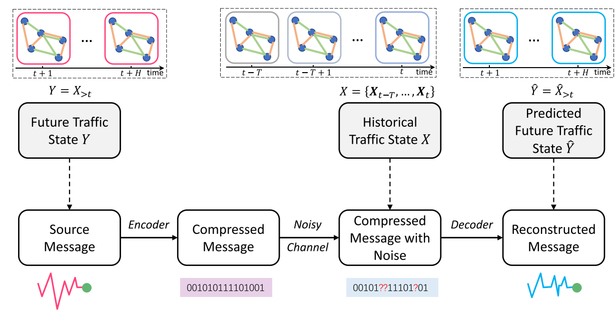


Fig. 2. Illustration of modeling traffic prediction as a transmission process over a noisy channel.

To analyze the underlying sources of prediction errors in the traffic prediction problem, we adopt the noisy channel model from information theory as follows:

$$Y \xrightarrow{\text{Encoder}} \tilde{X} \xrightarrow{\text{Noisy Channel}} X \xrightarrow{\text{Decoder}} \hat{Y}. \tag{2}$$

It considers a sender holding a source message Y that the receiver cannot directly observe. From the receiver's perspective, there is intrinsic uncertainty in determining Y. When a communication channel is established and the sender transmits Y, this uncertainty can be reduced or eliminated. To minimize the amount of transmitted data, the sender applies a lossless compression algorithm to produce a compressed message $\tilde{X}$, which is then transmitted. The receiver decodes $\tilde{X}$ to reconstruct $\hat{Y}$.

From an information theory perspective, traffic prediction can be modeled as message transmission over a noisy channel. This analogy is summarized in Table I and visually illustrated in Fig. 2. In (2), the decoding step corresponds to the traffic prediction process, where the Decoder represents the prediction function defined in (1).

With the help of the noisy channel model, the prediction errors $Y \neq \hat{Y}$ in traffic prediction arise from two sources:

- *Noisy Channel:* Channel noise results in information loss for X compared to its original state $\tilde{X}$, preventing the decoder from fully recovering Y. This type of error reflects the information missing from historical traffic data X relative to future traffic data Y and is termed ***Data-induced Errors***.
- *Decoder Capability:* Even if X retains complete information, the decoder may still fail to accurately recover Y due to limited decoding capability. Since the decoder corresponds to the prediction functions $f(\cdot)$, this error is determined by the prediction model's capacity and is referred to as ***Model-induced Errors***.

Remark 1: In the prediction model, the objective of parameter optimization is to minimize model-induced errors. However, even if model-induced errors are reduced to zero, data-induced errors remain, representing an intrinsic performance upper bound for the prediction model on a given traffic dataset. This upper bound is a dataset-specific feature and is not affected by the parameters of prediction models. Consequently, data-induced errors serve as a measurement of dataset prediction difficulty, i.e., large data-induced errors indicate a challenging dataset, and smaller errors imply an easier one. In the following section, we derive a sufficient condition to ensure that the data-induced error in the decomposed prediction problem is lower than that in the original problem, i.e., the sufficient condition for making the decomposed prediction problem easier than the original.

B. Fano's Inequality and Data-Induced Error Rate

In this section, we apply Fano's Inequality from information theory to derive a measure for data-induced errors.

Error Rate and Data-induced Error: We first give two concepts about prediction errors:

1) *Prediction Error Rate:* We assume the traffic state Y is a discrete variable with M possible values.[1] The *Error Rate* of the Markov chain in (2) is defined as

$$e = \Pr(\hat{Y} \neq Y). \tag{3}$$

This error rate is an end-to-end measure of prediction error, including both data-induced and model-induced errors.

2) *Entropy of Data-induced Error:* As mentioned in Section III-A, data-induced error represents the information loss from the future traffic state Y relative to historical traffic X. Thus, data-induced errors can be expressed as the uncertainty of Y given X, measured by the conditional entropy of Y given X, i.e.,

$$\mathrm{H}(\text{Data-induced Errors}) = \mathrm{H}(Y|X), \tag{4}$$

where $\mathrm{H}(\cdot)$ denotes entropy. Although $\mathrm{H}(Y|X)$ exclusively represents data-induced errors, it cannot be directly calculated due to the unknown distributions of Y and X.

Then, we adopt Fano's inequality [43] in information theory to build a bridge between the prediction error rate and the entropy of data-induced errors.

Theorem 1 (Fano's Inequality): Let $Y \to X \to \hat{Y}$ be a Markov chain, where Y and $\hat{Y}$ are discrete random variables with M different values. Let $e = \Pr(\hat{Y} \neq Y)$, we have the inequality to describe the relations between e and $\mathrm{H}(Y|X)$ as

$$\mathrm{H}(Y|X) \leq \mathrm{H}(e) + e\log(M-1), \tag{5}$$

Here, $\mathrm{H}(e)$ is the entropy of the error rate, which is calculated as

$$\mathrm{H}(e) = -e \cdot \log e - (1-e) \cdot \log(1-e). \tag{6}$$

Equation (5) is *Fano's inequality*, with its proof provided in Appendix A.1. Here, the error rate e and the entropy of data-induced error $\mathrm{H}(Y|X)$ are related through an inequality.

Lower Bound of Error Rate: Based on Fano's inequality, we can derive a lower bound for the error rate. We first define a variable S as

$$S(e) = \mathrm{H}(e) + e\log(M-1), \tag{7}$$

[1]This assumption can be satisfied through discretization for datasets with continuous values.

which is a function of the prediction error rate e. Considering the monotonicity of the function in (7), we present the following lemma.

Lemma 1: The function $S(e)$ is monotonically increasing when the condition $e < 1 - 1/M$ holds.

It is easily to prove that $\mathrm{d}^2 S(e)/\mathrm{d}e < 0$, and $S(e)$ is strictly concave function. The detailed proof is given in Appendix A.2. Since Y has M distinct values, the error rate e cannot exceed $1 - 1/M$, allowing us to directly treat (7) as a monotonically increasing function.

For the sake of argument, let's define $S^{\min} = \mathrm{H}(Y|X)$. According to Fano's inequality in (5), i.e., $\mathrm{H}(Y|X) \leq S(e)$, $S^{\min}$ serves as a lower bound for the function $S(e)$. Since $S(e)$ is monotonically increasing, $S^{\min}$ uniquely corresponds to a lower bound on the error rate, denoted as $e^{\min}$, which is implicitly defined by

$$S^{\min} = S(e^{\min}) = \mathrm{H}(e^{\min}) + e^{\min}\log(M-1). \tag{8}$$

Data-induced Error Rate: Based on (8), there exists a monotonically increasing implicit function $e^{\min} = f^{-1}(S^{\min})$. Here, $e^{\min}$ serves as a lower bound on the error rate, i.e., $e \geq e^{\min}$. Moreover, $e^{\min}$ is solely determined by the entropy of the data-induced errors, indicating that $e^{\min}$ represents the data-induced component of the overall prediction error rate e. We refer to $e^{\min}$ as the *Data-induced Error Rate*. This offers a measure of the prediction error rate attributable to data-induced errors.

C. Data-Induced Error Rates in Decomposition

In this section, we analyze the relationships between data-induced errors in an original prediction problem and its decomposed subproblems. We first give a formal definition of the data decomposition algorithm.

Definition 1 (Decomposition Algorithm): Given input and output data variables X and Y for a traffic prediction problem, a decomposition algorithm is defined as a function $f(\cdot)$ such that $\{X_1, \ldots, X_k, \ldots, X_K\} = f(X)$ and $\{Y_1, \ldots, Y_k, \ldots, Y_K\} = f(Y)$, where X_k and Y_k represent the k-th decomposed components. The decomposition algorithm ensures that $\{X_1, \ldots, X_K\}$ and $\{Y_1, \ldots, Y_K\}$ can collectively reconstruct X and Y, respectively. This process of data decomposition and reconstruction forms the Markov chains: $X \to \{X_1, \ldots, X_K\} \to X$ and $Y \to \{Y_1, \ldots, Y_K\} \to Y$.

Based on this definition, we derive the following lemma.

Lemma 2: Given the Markov chain $X \to \{X_1, \ldots, X_K\} \to X$ for a decomposition, we have $\mathrm{H}(X) = \mathrm{H}(X_1, \ldots, X_K)$. Similarly, $\mathrm{H}(Y) = \mathrm{H}(Y_1, \ldots, Y_K)$.

The proof of this lemma is provided in Appendix A.3. Based on Lemma 2, $S^{\min}$ can be expressed as

$$\begin{aligned} S^{\min} &= \mathrm{H}(Y|X) = \mathrm{H}\left(Y_1, \ldots, Y_K | X\right) \\ &= \mathrm{H}\left(Y_1, \ldots, Y_K | X_1, \ldots, X_K\right). \end{aligned} \tag{9}$$

For the k-th subproblem, we also have the noise channel transmission Markov chain $Y_k \to X_k \to \hat{Y}_k$. Let $S_k^{\min} = \mathrm{H}(Y_k|X_k)$. According to (8), there is a data-induced error rate $e_k^{\min}$ for the subproblem, which is an implicit function of $S_k^{\min}$. For $e_k^{\min}$ of the subproblem and $e^{\min}$ of the original problem, we present the following theorem.

Theorem 2 (Condition for $e^{\min} \geq e_k^{\min}$): Let $Y \in \{1, \ldots, M\}$ and $Y_k \in \{1, \ldots, M\}$ for all $k \in \{1, \ldots, K\}$. Suppose X and X_k are generated from the processes $Y \to X$ and $Y_k \to X_k$, respectively, and that X, Y, X_k, Y_k satisfy (9). Then, the inequality $e^{\min} \geq e_k^{\min}$ holds if $Y_k \perp X_{k'}$ for all $k' \neq k$.

Moreover, we have a more stringent theorem that can guarantee $e_k^{\min}$ is strictly less than $e^{\min}$ in the following.

Theorem 3 (Condition for $e^{\min} > e_k^{\min}$): For Theorem 2, if $Y_k \perp Y_{k'}$ for all $k' \neq k$, then the inequality $e^{\min} > e_k^{\min}$ holds.

The proofs of Theorems 2 and 3 are presented in Appendix A.5 and A.6, respectively.

Remark 2: Theorem 3 provides a sufficient condition for $e^{\min} > e_k^{\min}$, i.e.,, $Y_k \perp X_{k'}$ and $Y_k \perp Y_{k'}$ for all $k' \neq k$. This implies that if the decomposition algorithm ensures that Y_k for a subproblem is independent of the data in other subproblems, then the prediction difficulty (measured by the data-induced error rates) for *each* subproblem is lower than that of the original problem.

D. Data-Induced Error Values in Decomposition

Section III-C establishes the relationships between the data-induced error rates of the original problem and each subproblem. However, two issues remain to be addressed. First, as defined in Definition 1, the final prediction of the original problem is achieved by integrating the results from all subproblems. Therefore, we must compare the data-induced error between the original problem and the integrated prediction of all subproblems. Second, while the prediction error rate treats traffic prediction as a discrete problem, real-world applications often consider it as a continuous problem. We address both issues in this section.

1) Prediction Error Value: For a continuous prediction problem with a predicted traffic state $\hat{Y} \in \mathbb{R}$ and ground truth $Y \in \mathbb{R}$, the *error value* is defined as:

$$\xi = \mathbb{E}\left[\left(Y - \hat{Y}\right)^2\right]. \tag{10}$$

In the predictive learning framework [81], prediction errors are divided into *error certainty* and *error magnitude*. Error certainty refers to the probability of a prediction error occurring, denoted as the error rate $e = \Pr(Y \neq \hat{Y})$, while error magnitude represents the size of the error. Let $Y^{(e)}$ and $\hat{Y}^{(e)}$ be the variables corresponding to Y and $\hat{Y}$ when $Y \neq \hat{Y}$. The error magnitude is then defined as:

$$\mathbb{E}[\delta] = \mathbb{E}\left[\left(Y^{(e)} - \hat{Y}^{(e)}\right)^2\right]. \tag{11}$$

Using the error certainty e and error magnitude $\mathbb{E}[\delta]$, the prediction error value ξ can be expressed as:

$$\xi = e \cdot \mathbb{E}[\delta]. \tag{12}$$

2) Data-induced Error Magnitudes: For error magnitudes, we assume they consist of both a model-induced and a data-induced component. Thus, the error magnitude in (11) can be expressed as:

$$\mathbb{E}[\delta] = \mathbb{E}\left[\left(Y^{(e)} - \left(Y^{(e)} + \Delta^{\min} + \Delta^{\text{model}}\right)\right)^2\right], \tag{13}$$

where $\Delta^{\min}$ and Δ^{model} represent the data-induced and model-induced components for error magnitudes, respectively. Therefore, the *Data-induced Error Magnitudes* for an original problem can be expressed as:

$$\mathbb{E}[\delta^{\min}] = \mathbb{E}\left[\left(Y^{(e)} - \left(Y^{(e)} + \Delta^{\min}\right)\right)^2\right] = \mathbb{E}\left[\left(\Delta^{\min}\right)^2\right]. \tag{14}$$

Assuming the decomposition algorithm cannot reduce the data-induced error magnitudes, the data-induced components for error magnitudes of all decomposed predictions satisfy $\Delta^{\min} = \sum_{k=1}^{K} \Delta_k^{\min}$, where $\Delta_k^{\min}$ is the data-induced components for error magnitude of the k-th decomposition. The data-induced error magnitudes for the k-th decomposed subproblem is defined as $\mathbb{E}[\delta_k^{\min}] = \mathbb{E}[(\Delta_k^{\min})^2]$. Based on the above assumption, we derive the following lemma.

Lemma 3: Given mutually independent decomposed components $\{Y_1, \ldots, Y_K\}$, if variables $\{X_1, \ldots, X_K\}$ are also mutually independent, then $\mathbb{E}[\delta^{\min}] = \sum_{k=1}^{K} \mathbb{E}[\delta_k^{\min}]$, where $\mathbb{E}[\delta_k^{\min}]$ is the data-induced error magnitude for the k-th decomposed subproblem.

The proof of this lemma is provided in Appendix A.4.

3) Data-induced Error Value: For a given prediction problem, the *Data-induced Error Value* for the original problem and its decomposed subproblems are defined as:

$$\xi^{\min} = e^{\min} \cdot \mathbb{E}[\delta^{\min}], \quad \xi_k^{\min} = e_k^{\min} \cdot \mathbb{E}[\delta_k^{\min}], \tag{15}$$

where $\xi^{\min}$ represents the error value for the original problem, and $\xi_k^{\min}$ represents the error value for the k-th subproblem. The relationship between the data-induced error value $\xi^{\min}$ for the original problem and the integrated data-induced error value $\sum_{k=1}^{K} \xi_k^{\min}$ for all of the decomposed subproblems is established in the following theorem.

Theorem 4 (Condition for $\xi^{\min} > \sum_{k=1}^{K} \xi_k^{\min}$): Let $Y \in \{1, \ldots, M\}$, $Y_k \in \{1, \ldots, M\}$, and $\hat{Y}_k \in \{1, \ldots, M\}$ for all $k \in \{1, \ldots, K\}$, and let X and X_k be variables generated from the Markov chains $Y \to X$ and $Y_k \to X_k$, respectively. Assume that X, Y, X_k, Y_k satisfy (9). The inequality $\xi^{\min} > \sum_{k=1}^{K} \xi_k^{\min}$ holds if the following conditions are satisfied:

1) For any Y_k, $Y_k \perp X_{k'}$ for all $k' \neq k$;
2) For any pair Y_k and $Y_{k'}$ in $\{Y_1, \ldots, Y_K\}$, $Y_k \perp Y_{k'}$;
3) For any pair X_k and $X_{k'}$ in $\{X_1, \ldots, X_K\}$, $X_k \perp X_{k'}$.

Proof: Using (15), the data-induced total error value for all of the subproblems can be expressed as:

$$\begin{aligned}\sum_{k=1}^{K} \xi_k^{\min} &= \sum_{k=1}^{K} e_k^{\min} \cdot \mathbb{E}[\delta_k^{\min}] \\ &< \sum_{k=1}^{K} e^{\min} \cdot \mathbb{E}[\delta_k^{\min}] = e^{\min} \sum_{k=1}^{K} \mathbb{E}[\delta_k^{\min}],\end{aligned} \tag{16}$$

where the inequality applies Theorem 3, which states that $e_k^{\min} < e^{\min}$ if conditions 1) and 2) are satisfied.

From condition 3), $\{X_1, \ldots, X_K\}$ are mutually independent. According to Lemma 3, this implies that:

$$\mathbb{E}[\delta^{\min}] = \sum_{k=1}^{K} \mathbb{E}[\delta_k^{\min}]. \tag{17}$$

Substituting this equation into (16), we get:

$$e^{\min}\sum_{k=1}^{K}\mathbb{E}[\delta_k^{\min}] = e^{\min}\cdot\mathbb{E}[\delta^{\min}] = \xi^{\min}. \tag{18}$$

From (16) and (18), it follows that:

$$\xi^{\min} > \sum_{k=1}^{K}\xi_k^{\min}. \tag{19}$$

Thus, the theorem is proved. □

Theorem 4 establishes the relationship between the data-induced error in the original problem and that in its decomposition (integrated prediction across subproblems) for a continuous prediction problem, effectively addressing the two unresolved issues.

E. Component Independence Principle

According to Theorem 4, if a decomposition algorithm yields components satisfying $Y_k \perp X_{k'}$, $Y_k \perp Y_{k'}$, and $X_k \perp X_{k'}$ for all $k' \neq k$, the total data-induced error in the decomposed prediction is no greater than that of the original prediction. This result indicates that enforcing independence among decomposed components provides a sufficient condition for reducing the intrinsic difficulty of the prediction problem. Under the idealized assumption of a sufficiently expressive prediction model that can eliminate model-induced errors, such a decomposition can in principle lead to improved prediction performance compared to the undecomposed setting.

In a traffic prediction task, the variables X and Y denote the traffic state time series, i.e., $\{X^{(1)},\ldots,X^{(t)},Y^{(t+1)}\}$, where $X^{(t)}$ represents the observed traffic state at time t. To satisfy the sufficient conditions stated in Theorem 4, a decomposition algorithm needs to encourage statistical independence between the decomposed sequences $\{X_k^{(1)},\ldots,X_k^{(t)},Y_k^{(t+1)}\}$ and $\{X_{k'}^{(1)},\ldots,X_{k'}^{(t)},Y_{k'}^{(t+1)}\}$ for all $k \neq k'$. This observation motivates the following design guideline for decomposition algorithms:

- *An effective decomposition algorithm may benefit from promoting independence among decomposed components.*

We refer to this guideline as the *Component Independence (CI) Principle*, which serves as a sufficient and theoretically grounded design principle for the development of decomposition-based prediction models in the subsequent sections.

Remark 3: The CI Principle is established as a sufficient condition for reducing data-induced uncertainty, rather than a necessary or exhaustive characterization of all effective decomposition strategies. Decompositions that do not strictly satisfy the CI Principle may still be beneficial in practice, but they fall outside the scope of the current theoretical analysis. Nevertheless, by identifying independence as a sufficient mechanism for uncertainty reduction, the CI Principle provides a useful and theoretically grounded guideline for the design of decomposition algorithms.

IV. The TGDL Model

In this section, we propose a Theory-guided Graph Decomposition Learning (TGDL) framework, grounded in the CI principle outlined in Section III. The CI Principle states that decomposing a traffic prediction problem into mutually independent subcomponents can effectively reduce data-induced errors and thus improve predictability. This provides a theoretical foundation for designing decomposition-based learning frameworks. Guided by this principle, our TGDL framework is designed to produce traffic subgraphs that are as independent as possible in both structure and signal. This theoretical goal is achieved through two decomposition modules and two regularization terms:

- *Structure Decomposition Module (Section IV-B)* partitions the original traffic graph into multiple structurally independent subgraphs. The corresponding *Structure Independence Regularization* (25) penalizes structural overlaps among subgraphs by minimizing the pairwise Frobenius inner product of their adjacency matrices.
- *Signal Decomposition Module (Section IV-C)* disentangles the dynamic graph signals into multiple temporal components aligned with the subgraphs. The *Signal Independence Regularization* (34) minimizes mutual information between the extracted representations of signal components.

A. Preliminaries

The TGDL framework organizes traffic state data as a graph with dynamic signals, referred to as the *Traffic Graph*. The structure of the traffic graph is defined as follows:

Definition 2 (Structure of Traffic Graph): The structure of a traffic graph is denoted by $G = (\mathcal{V}, \boldsymbol{A})$, where $\mathcal{V} = \{v_1,\ldots,v_I\}$ is the set of vertices, with each vertex representing a geographical unit (e.g., a road segment or land parcel). The adjacency matrix $\boldsymbol{A}\in\mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ contains the edge weights, where the (i,j)-th element, denoted as a_{ij}, represents the weight of the edge from vertex v_i to vertex v_j.

Edge weights can be calculated using various methods, such as geographic distance [19], [21], correlation between vertex features [82], [83], or other relevant metrics.

For a traffic graph, the dynamic traffic state is represented as graph signals.

Definition 3 (Traffic Graph Signals): The timeline is discretized into time slots. Given a traffic graph G, the signal of vertex v_i at time slot t is the vector $\boldsymbol{x}_{i,t}\in\mathbb{R}^F$, where the elements of $\boldsymbol{x}_{i,t}$ represent the traffic states, F is the number of traffic state channels (e.g., inflow, outflow, speed). The signal of entire graph at time t is the matrix $\boldsymbol{X}_t\in\mathbb{R}^{|\mathcal{V}|\times F}$, where the i-th row is $\boldsymbol{x}_{i,t}$.

Original Prediction Problem: Given a traffic graph G, its historical signals over a time window $\{t-T,\ldots,t\}$ are denoted as $\boldsymbol{\mathcal{X}} = \{\boldsymbol{X}_{t-T},\ldots,\boldsymbol{X}_t\}$. The future traffic states for time slots $> t$ are denoted as $\boldsymbol{Y}_{>t}$. The original traffic state prediction problem is formulated as

$$\hat{\boldsymbol{Y}}_{>t} = f_g\left(\boldsymbol{\mathcal{X}}_t, G, \boldsymbol{\Theta}\right), \tag{20}$$

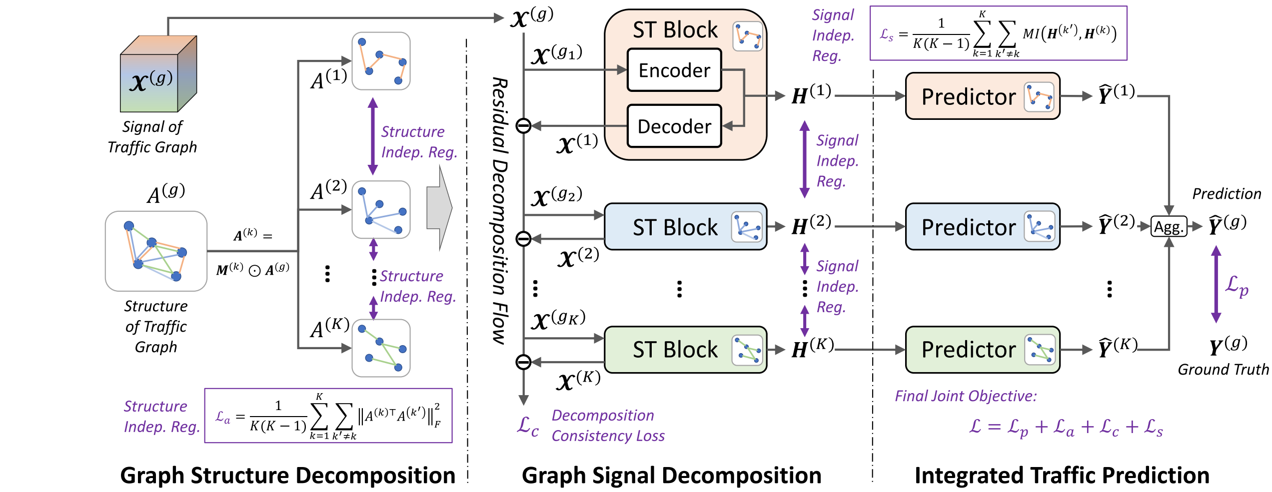


Fig. 3. The pipeline of our TGDL framework. Indep: Independence. Reg: Regularization. Agg: Aggregation.

where $\hat{\boldsymbol{Y}}_{>t}$ represents the predicted future traffic states, and $\boldsymbol{\Theta}$ are the learnable parameters.

Decomposition Prediction: Given an original traffic graph $G^{(g)} = (\mathcal{V}, \boldsymbol{A}^{(g)})$ with historical signals $\boldsymbol{\mathcal{X}}_t^{(g)}$, a decomposition algorithm divides the original graph into K subgraph components, denoted as $\{G^{(1)}, \ldots, G^{(K)}\}$, with corresponding signals $\{\boldsymbol{\mathcal{X}}_t^{(1)}, \ldots, \boldsymbol{\mathcal{X}}_t^{(K)}\}$. These subgraphs satisfy

$$\boldsymbol{A}^{(g)} = \sum_{k=1}^{K} \boldsymbol{A}^{(k)} \quad \text{and} \quad \boldsymbol{\mathcal{X}}_t^{(g)} = \sum_{k=1}^{K} \boldsymbol{\mathcal{X}}_t^{(k)}, \tag{21}$$

where $\boldsymbol{A}^{(k)}$ is the adjacency matrix of the k-th subgraph. The decomposition prediction problem is then defined as

$$\hat{\boldsymbol{Y}}_{>t}^{(k)} = f_d\left(\boldsymbol{\mathcal{X}}_t^{(k)}, G^{(k)}, \boldsymbol{\Theta}^{(k)}\right) \quad s.t. \quad \hat{\boldsymbol{Y}}_{>t}^{(g)} = \sum_{k=1}^{K} \hat{\boldsymbol{Y}}_{>t}^{(k)}, \tag{22}$$

where $\boldsymbol{\Theta}^{(k)}$ is the parameters of the k-th prediction function.

The TGDL framework implements decomposition prediction through three steps (see Fig. 3): Graph Structure Decomposition, Graph Signal Decomposition, and Integrated Traffic Prediction.

B. Graph Structure Decomposition

The TGDL framework first applies a *Structure Independent Decomposition* (SID) to partition the structure of the original graph $G^{(g)}$ into subgraphs. For the k-th subgraph, the vertex set remains the same as the original graph, while the adjacency matrix $\boldsymbol{A}^{(k)}$ for each subgraph is calculated as

$$\boldsymbol{A}^{(k)} = \boldsymbol{M}^{(k)} \odot \boldsymbol{A}^{(g)}, \tag{23}$$

where $\odot$ denotes the Hadamard product, and $\boldsymbol{M}^{(k)}$ is a mask matrix with elements $m_{ij} \in [0, 1]$ representing the interaction between nodes in the k-th subgraph.

To ensure that the summation of the subgraph adjacency matrices equals the original graph, i.e., satisfying (21), we use a SoftMax function to calculate m_{ij} as

$$m_{ij} = \frac{\exp\left(w_{ij}^{(k)}\right)}{\sum_{k'=1}^{K} \exp\left(w_{ij}^{(k')}\right)}, \tag{24}$$

where $w_{ij}^{(k)}$ are learnable parameters. This formulation ensures that $\sum_{k=1}^{K} m_{ij}^{(k)} = 1$ in the mask matrix, and consequently, $\sum_{k=1}^{K} a_{ij}^{(k)} = a_{ij}^{(g)}$ for each edge.

According to the CI Principle derived in Section III, an effective decomposition algorithm should ensure that the decomposed subgraph components are mutually independent. To this end, we introduce a *Structure Independence Regularization* $\mathcal{L}_a$ to penalize correlations among the adjacency matrices of different subgraphs:

$$\mathcal{L}_a = \frac{1}{K(K-1)} \sum_{k=1}^{K} \sum_{k' \neq k} \left\| \boldsymbol{A}^{(k)\top} \boldsymbol{A}^{(k')} \right\|_F^2, \tag{25}$$

where $\|\cdot\|_F$ is the Frobenius norm. Minimizing $\mathcal{L}_a$ encourages these adjacency matrices to be approximately orthogonal, thereby promoting the generation of non-overlapping and minimally interfering subgraph structures.

Remark 4: Our proposed regularization explicitly promotes structural decoupling by discouraging shared components between subgraphs, thereby encouraging the decomposition to satisfy the CI Principle. To illustrate this point even further, we propose a theoretical property of the *Structure Independence Regularization* (see Sect. III.C of the Supplementary Materials for details).

C. Graph Signal Decomposition

Next, the TGDL model employs a *Residual Decomposition Flow* (RDF) to iteratively extract independent signal components for each subgraph structure from the original graph.

1) Residual Decomposition Flow: Let $\boldsymbol{\mathcal{X}}_t^{(g)}$ denote the signal of the original graph at time t. For the first subgraph $G^{(1)}$, its signal at t is extracted from $\boldsymbol{\mathcal{X}}_t^{(g)}$ using a *spatiotemporal block* (*ST block*) as:

$$\boldsymbol{\mathcal{X}}_t^{(1)} = \text{STBlock}\left(\boldsymbol{\mathcal{X}}_t^{(g)}, \boldsymbol{A}^{(1)}, \boldsymbol{\Theta}_b^{(1)}\right), \tag{26}$$

where $\boldsymbol{\mathcal{X}}_t^{(1)} = \{\boldsymbol{X}_{t-T}^{(1)}, \ldots, \boldsymbol{X}_t^{(1)}\}$ is the signal of $G^{(1)}$, and $\boldsymbol{\Theta}_b^{(1)}$ denotes the learnable parameters. The function STBlock$(\cdot)$ is a graph neural network, detailed in Section IV-C2.

Subsequently, we remove the signal $\boldsymbol{\mathcal{X}}_t^{(1)}$ from the original graph signal as

$$\boldsymbol{\mathcal{X}}_t^{(g_2)} = \boldsymbol{\mathcal{X}}_t^{(g)} - \boldsymbol{\mathcal{X}}_t^{(1)}. \tag{27}$$

Here, $\boldsymbol{\mathcal{X}}_t^{(g_2)}$ represents the residual signal after subtracting the signal of $G^{(1)}$ from the original signal. The ST block is then used to extract the signal of $G^{(2)}$ from $\boldsymbol{\mathcal{X}}_t^{(g_2)}$ as

$$\boldsymbol{\mathcal{X}}_t^{(2)} = \text{STBlock}\left(\boldsymbol{\mathcal{X}}_t^{(g_2)}, \boldsymbol{A}^{(2)}, \boldsymbol{\Theta}_b^{(2)}\right). \tag{28}$$

Given the K subgraphs, we iteratively apply (26) and (27) to construct a *residual decomposition flow* (RDF), where the input signal for the k-th subgraph is calculated as

$$\boldsymbol{\mathcal{X}}_t^{(g_k)} = \boldsymbol{\mathcal{X}}_t^{(g_{k-1})} - \boldsymbol{\mathcal{X}}_t^{(k-1)}. \tag{29}$$

The signal component for the k-th subgraph is

$$\boldsymbol{\mathcal{X}}_t^{(k)} = \text{STBlock}\left(\boldsymbol{\mathcal{X}}_t^{(g_k)}, \boldsymbol{A}^{(k)}, \boldsymbol{\Theta}_b^{(k)}\right). \tag{30}$$

Here, we set $\boldsymbol{\mathcal{X}}_t^{(g_1)} = \boldsymbol{\mathcal{X}}_t^{(g)}$ to maintain notational consistency. Through the algorithmic flow defined by (29) and (30), the original signal $\boldsymbol{\mathcal{X}}_t^{(g)}$ is decomposed into a series of subgraph signals, i.e., $\{\boldsymbol{\mathcal{X}}_t^{(1)}, \dots, \boldsymbol{\mathcal{X}}_t^{(K)}\}$.

According to the definition of decomposition prediction in (21), the sum of the subgraph signals should equal the original signal. Thus, we introduce a *Decomposition Consistency Loss* to constrain the subgraph signals:

$$\mathcal{L}_c = \left\| \boldsymbol{\mathcal{X}}_t^{(g)} - \sum_{k=1}^{K} \boldsymbol{\mathcal{X}}_t^{(k)} \right\|_F^2. \tag{31}$$

In the RDF process described in (29), each subgraph's signal component is sequentially removed from the original signal. Consequently, the subgraph signals extracted by (30) are decoupled from one another, partially satisfying the CI principle.

2) Spatiotemporal Blocks: The TGDL framework implements the ST block (i.e., STBlock$(\cdot)$ function) in (30) as a neural network with an encoder-decoder structure. Specifically, given the residual graph signal $\boldsymbol{\mathcal{X}}_t^{(g_k)}$ for the k-th subgraph, the ST block uses an encoder to extract signal representations as

$$\boldsymbol{H}_t^{(k)} = \text{Encoder}\left(\boldsymbol{\mathcal{X}}_t^{(g_k)}, \boldsymbol{A}^{(k)}, \boldsymbol{\Theta}_e^{(k)}\right), \tag{32}$$

where $\boldsymbol{H}_t^{(k)} \in \mathbb{R}^{|\mathcal{V}| \times D}$ is the matrix of signal representations for subgraph k. Each row $\boldsymbol{h}_i^{(k)} \in \mathbb{R}^D$ corresponds to the representation of vertex v_i, with D as the dimension of the representation vectors. $\boldsymbol{\Theta}_e^{(k)}$ denotes the learnable parameters of the encoder.

Next, for the k-th subgraph signals, the ST block adopts a decoder to covert the representations $\boldsymbol{H}^{(k)}$ as

$$\boldsymbol{\mathcal{X}}_t^{(k)} = \text{Decoder}\left(\boldsymbol{H}_t^{(k)}, \boldsymbol{A}^{(k)}, \boldsymbol{\Theta}_d^{(k)}\right), \tag{33}$$

where $\boldsymbol{\mathcal{X}}_t^{(k)}$ is the decomposed signal of the subgraph $G^{(k)}$, and $\boldsymbol{\Theta}_d^{(k)}$ is the learnable parameter for the decoder. The encoder in (32) and the decoder in (33) form the STBlock$(\cdot)$ function in (30). The encoder and decoder functions can be implemented by any encoder-decoder models that are designed to handle graph-based traffic data [15], [21], [84].

3) Signal Independence Regularization: According to the CI principle in Theorem 4, the signal components of different subgraphs should be mutually independent. In the algorithm flow formed by (29) and (30), the original signal is processed into a series of decoupled subgraph signals. To further enforce the independence between subgraph signals, we introduce a *Signal Independence Regularization* (SIR) to constrain the signal representation matrices generated by the ST blocks.

Given the signal representation matrices $\boldsymbol{H}^{(k)}$ for $k \in \{1, \dots K\}$ generated by (32), the SIR is defined as

$$\mathcal{L}_s = \frac{1}{K(K-1)} \sum_{k=1}^{K} \sum_{k' \neq k} \text{MI}\left(\boldsymbol{H}^{(k')}, \boldsymbol{H}^{(k)}\right), \tag{34}$$

where $\text{MI}(\boldsymbol{H}^{(k')}, \boldsymbol{H}^{(k)})$ represents the *Mutual Information* (MI) between the signal representations of subgraphs k' and k, defined as

$$\text{MI}\left(\boldsymbol{H}^{(k')}, \boldsymbol{H}^{(k)}\right) = \mathbb{E}\left[\log \frac{\Pr\left(\boldsymbol{H}^{(k')}, \boldsymbol{H}^{(k)}\right)}{\Pr\left(\boldsymbol{H}^{(k')}\right) \cdot \Pr\left(\boldsymbol{H}^{(k)}\right)}\right], \tag{35}$$

where $\Pr(\boldsymbol{H}^{(k)})$, $\Pr(\boldsymbol{H}^{(k')})$, and $\Pr(\boldsymbol{H}^{(k')}, \boldsymbol{H}^{(k)})$ correspond to the marginal and joint distributions of $\boldsymbol{H}^{(k)}$ and $\boldsymbol{H}^{(k')}$.

Due to the intractability of the closed-form expressions for the joint and marginal distributions of $\boldsymbol{H}^{(k')}$ and $\boldsymbol{H}^{(k)}$, direct computation of the MI in (35) is not feasible. As an alternative, we approximate the MI using the CLUB method [85]. Specifically, we compute an upper bound on the MI between $\boldsymbol{H}^{(k')}$ and $\boldsymbol{H}^{(k)}$ as

$$\begin{aligned} \widehat{\text{MI}}_{\text{UB}}\left(\boldsymbol{H}^{(k')}, \boldsymbol{H}^{(k)}\right) &= \frac{1}{N} \sum_{n=1}^{N} \left[\log q_\theta\left(\boldsymbol{H}_n^{(k')} | \boldsymbol{H}_n^{(k)}\right)\right. \\ &\quad \left. - \frac{1}{N} \sum_{n'=1}^{N} \log q_\theta\left(\boldsymbol{H}_{n'}^{(k')} | \boldsymbol{H}_n^{(k)}\right)\right], \end{aligned} \tag{36}$$

where N is the number of samples, and $\boldsymbol{H}_i^{(k)}$ and $\boldsymbol{H}_i^{(k')}$ denote the i-th sample of the signal representations for subgraphs k and k', respectively. The $q_\theta(\boldsymbol{H}^{(k')} | \boldsymbol{H}^{(k)})$ in (36) represents a variational approximation of the conditional probability $\Pr(\boldsymbol{H}^{(k')} | \boldsymbol{H}^{(k)})$, assumed to follow a Gaussian distribution $\mathcal{N}(\mu_{\boldsymbol{H}^{(k)}}, \sigma^2_{\boldsymbol{H}^{(k)}})$, i.e.,

$$q_\theta\left(\boldsymbol{H}^{(k')} | \boldsymbol{H}^{(k)}\right) = -\frac{1}{\sqrt{2\pi}\sigma_{\boldsymbol{H}^{(k)}}} \exp\left(-\frac{(x - \mu_{\boldsymbol{H}^{(k)}})^2}{2\sigma^2_{\boldsymbol{H}^{(k)}}}\right). \tag{37}$$

here, the mean $\mu_{\boldsymbol{H}^{(k)}}$ and variance $\sigma^2_{\boldsymbol{H}^{(k)}}$ are estimated using a multi-layer perceptron (MLP) network with $\boldsymbol{H}^{(k)}$ as input, i.e.,

$$\left\{\mu_{\boldsymbol{H}^{(k)}}, \sigma^2_{\boldsymbol{H}^{(k)}}\right\} = \text{MLP}\left(\boldsymbol{H}^{(k)}, \boldsymbol{\Theta}_{\text{mlp}}\right), \tag{38}$$

where $\boldsymbol{\Theta}_{\text{mlp}}$ represents the learnable parameters of the MLP.

Finally, we substitute $\widehat{\text{MI}}_{\text{UB}}(\boldsymbol{H}^{(k')}, \boldsymbol{H}^{(k)})$ from (36) into (34) to implement the signal independence regularization. By minimizing the mutual information between $\boldsymbol{H}^{(k')}$ and $\boldsymbol{H}^{(k)}$, our objective is to promote the independence of signals within subgraphs k and k'. This ensures that the RDF aligns with the decomposition independence principle outlined in Theorem 4.

Remark 5: The SIR $\mathcal{L}_s$ is minimized when the mutual information term in (34) is minimized. This indicates that each subcomponent fully discards the correlated part, thereby enforcing the CI Principle. The point is proved in Sect. III.D of the Supplementary Materials.

D. Integrated Traffic Prediction

For each subgraph $G^{(k)}$, the traffic state is predicted using a decoder with the signal representations $\boldsymbol{H}^{(k)}$ as input, i.e.,

$$\hat{\boldsymbol{Y}}_{>t}^{(k)} = \text{Predictor}\left(\boldsymbol{H}_t^{(k)}, \boldsymbol{A}^{(k)}, \boldsymbol{\Theta}_p^{(k)}\right). \tag{39}$$

here, Predictor$(\cdot)$ is a prediction function, implemented similarly to the decoder in (33), but with distinct outputs and learnable parameters $\boldsymbol{\Theta}_p^{(k)}$. The final prediction of the original signal is obtained by aggregating the predicted subgraph signals via summation operation, i.e.,

$$\hat{\boldsymbol{Y}}_{>t}^{(g)} = \sum_{k=1}^{K} \hat{\boldsymbol{Y}}_{>t}^{(k)}. \tag{40}$$

The loss function for the final prediction is then defined as

$$\mathcal{L}_p = \left\|\boldsymbol{Y}_{>t} - \hat{\boldsymbol{Y}}_{>t}^{(g)}\right\|_F^2. \tag{41}$$

In the TGDL framework, both the decoder in (33) and the predictor in (39) share the same architecture but with distinct parameters. This encoder-decoder (predictor) pair can be implemented using any graph neural network-based traffic prediction model that adopts an encoder-decoder structure [15], [21], [84]. The model used for the encoder-decoder (predictor) implementation is referred to as the *base model* in the TGDL framework. Thus, TGDL can be viewed as a decomposition prediction framework that enhances the performance of various base models.

E. Model Training

The objective function for training the model consists of three components: i) the loss function $\mathcal{L}_p$ in (41) to minimize prediction error; ii) the decomposition consistency loss $\mathcal{L}_c$ in (31) to enforce subgraph signals to satisfy the decomposition prediction requirements; and iii) the independence regularization terms $\mathcal{L}_a$ and $\mathcal{L}_s$ in (25) and (34) to ensure the graph decomposition adheres to the CI principle outlined in Theorem 4. The final joint objective function is defined as

$$\mathcal{L} = \gamma_p \mathcal{L}_p + \gamma_a \mathcal{L}_a + \gamma_c \mathcal{L}_c + \gamma_s \mathcal{L}_s. \tag{42}$$

To avoid manually selecting hyperparameters, the weights $\gamma_p, \gamma_a, \gamma_c, \gamma_s$ are set through a Dynamic Weight Averaging (DWA) strategy [86], with an initial value of 1. DWA dynamically adjusts the weight of each loss term based on its change rate during training to balance the loss changes of all tasks. This prevents any single task from dominating, promoting balanced learning across all tasks.

V. Experiments

A. Experimental Settings

1) Data Description. Datasets: To evaluate the effectiveness of our proposed method, we conduct experiments on four real-world spatiotemporal traffic datasets, detailed as follows:

- *NYCBike:* A public traffic flow dataset collected in New York City, containing bike rental data from 04/01/2014 to 09/30/2014 [25]. We divide the city into a 16×8 grid and map bike rental records as traffic flow between zones. Traffic flows are measured hourly, totaling 4,392 samples.
- *NYCTaxi:* Another public dataset from New York City, containing taxi GPS data from 01/01/2015 to 03/01/2015. The city is divided into a 20×10 grid to map taxi trajectories as traffic flow between zones [45]. Traffic flows are recorded every 30 minutes, resulting in 2,880 samples.
- *PEMSD4:* A public traffic dataset collected by California's Performance of Transportation (PeMS) [87], comprising data from 307 sensors in the San Francisco Bay Area over two months (01/01/2018 to 02/28/2018) [88]. Traffic data, recorded every 5 minutes, is aggregated into 30-minute intervals, yielding 2,832 samples.
- *PEMSD8:* Another dataset from PeMS, consisting of data from 170 sensors in District 08 over two months (07/01/2016 to 08/31/2016) [88]. Originally recorded every 5 minutes, data is aggregated into 30-minute intervals, resulting in 2,976 samples.

For each prediction sample at time slot t, we use two types of historical data as inputs: i) data from the 4 hours preceding time t; ii) data from 2 hours before and after time slots $t - T_{\text{day}}$, $t - 2 \times T_{\text{day}}$, and $t - 3 \times T_{\text{day}}$, where T_{day} is the number of slots in a day. The second type incorporates periodicity information into the prediction. We use a sliding window strategy to generate samples, and each dataset is split into training, validation, and test sets in a 7:1:2 ratio.

Graph Structure Construction: The traffic graph structure is represented by an adjacency matrix $\boldsymbol{A}$. Let a_{ij} denote an entry in $\boldsymbol{A}$; following [19], we construct $\boldsymbol{A}$ using a thresholded Gaussian kernel:

$$a_{ij} = \begin{cases} \exp\left(-\frac{\mathcal{D}(v_i, v_j)^2}{\sigma^2}\right), & \mathcal{D}(v_i, v_j) \le r, \\ 0, & \mathcal{D}(v_i, v_j) > r. \end{cases} \tag{43}$$

where $\mathcal{D}(v_i, v_j)$ denotes the distance between nodes v_i and v_j, σ is the standard deviation of distances, and the threshold r is set to 0.6. For PEMSD4 and PEMSD8, we compute $\mathcal{D}(v_i, v_j)$ based on actual road distances between sensors. For NYCBike and NYCTaxi, we measure the distance using dynamic time warping (DTW) on the traffic time series between nodes, i.e., $\mathcal{D}(v_i, v_j) = \text{DTW}(\boldsymbol{x}_i, \boldsymbol{x}_j)$.

Task Description: For NYCBike and NYCTaxi, the datasets provide inflow and outflow counts for each region. This enables direct modeling of spatially distributed inflow/outflow patterns. Following prior works [15], [25], [45], we adopt a single-step prediction setting, where the model predicts both inflow and outflow at the next time step. For PEMSD4 and PEMSD8, the datasets contain aggregated traffic volume at each sensor location, without separate inflow and outflow information. Therefore, consistent with common practice in existing works [37], [88], [89], we adopt a multi-step prediction setting, where the model forecasts the total traffic flow from 30 minutes to 2 hours ahead (i.e., 1–4 future steps). To validate that our framework generalizes across different prediction settings, we have also included multi-step prediction results on the NYCBike and NYCTaxi datasets in Sect. II.F of the Supplementary Materials.

In theoretical derivation, we assume the traffic state Y is a discrete variable. Empirically, our experiment focuses on traffic flow data, whose smallest unit is 1. This allows them to be treated as discrete variables without further discretization, thus satisfying the "discrete value" assumption. For inherently continuous traffic states like speed, discretization is done based on practical needs, such as categorizing speed into levels for congestion scenarios [42].

2) Experiment Scenarios & Baselines. Experiments with Different Base-models: We employ two experiment scenarios to assess the performance of the proposed TGDL framework. In the first scenario, called *experiments with different base-models*, we use various GNN-based spatiotemporal prediction methods as the base model for TGDL, comparing the performance of TGDL with that of the original base models. This scenario not only verifies TGDL's effectiveness over the base models but also demonstrates its robustness across different base models. The baselines in this scenario include five state-of-the-art GNN-based spatiotemporal prediction models:

- *STGCN [84]:* It is a graph convolution-based model that uses 1D convolution to capture temporal correlations.
- *GWNet [21]:* This work combines diffusion graph convolution with dilated causal convolution to capture spatiotemporal dependencies.
- *MTGNN [89]:* This method integrates mix-hop graph convolution and temporal convolution for multivariate time series prediction.
- *MSDR [18]:* It is an RNN variant incorporating graph convolution in each time unit for spatiotemporal prediction, explicitly using hidden states from multiple past steps as input.
- *STSSL [15]:* This model is a self-supervised spatiotemporal prediction model, which designs two auxiliary tasks to learn effective hidden embeddings.

All baselines follow an encoder-decoder structure, serving as the base models for TGDL and providing a basis for comparison.

Comparison with Decomposition Baselines: The second experimental scenario is *comparison with decomposition baselines*. In this setting, we compare the TGDL model with decomposition-based prediction baselines across three categories. The first category is mainly two-stage decomposition prediction approaches.

- *PTD-XFMR [60]:* This is a classic two-stage method, where the decomposition is performed using Periodic-Trend Decomposition (PTD), and forecasting is conducted using sequence learning models such as LSTM. To enhance this model, we replace the LSTM with the powerful Transformer (XFMR) model.
- *EMD-XFMR [90]:* The decomposition step is performed by the two-level Empirical Mode Decomposition (EMD). The prediction step is realized by GRU, which is replaced with Transformer (XFMR) for stronger modeling capacity.

The second category includes state-of-the-art models with signal decomposition.

- *CoST [33]:* It uses contrastive learning to decompose seasonal-trend representations for time series forecasting.
- *D2STGNN [36]:* The model decomposes data into diffusion and non-diffusion signals for traffic prediction.
- *STWave [39]:* It separates complex traffic data into stable trends and fluctuating events for improved prediction.
- *SCNN [32]:* This work iteratively decomposes input data into long-term, seasonal, short-term, and co-evolving components for spatiotemporal prediction.

The third category comprises prediction models with both structure and signal decomposition. Since existing decomposition prediction models primarily focus on signal decomposition, we apply TGDL's signal decomposition alongside two state-of-the-art graph structure decomposition methods as baselines for this category.

- *TGDL-METIS:* This variant replaces the graph structure decomposition module in TGDL with METIS [63], a hierarchical, node-based graph decomposition algorithm.
- *TGDL-DBH:* The model replaces the graph structure decomposition module with degree-based hashing [64], an edge-based graph decomposition method.

Moreover, we include two types of classic non-deep models as a basic performance reference.

- *HA:* It models ST data as a seasonal process, predicting values by averaging historical data at the target time.
- *SVR:* A regression model using linear support vector machines, representing a classic non-deep learning approach.

3) Implementation Details: We implement our model using Python 3.9 and PyTorch 1.12.0. The model is trained with the Adam optimizer, with the learning rate set to 0.001. The number of subgraphs is optimized within the range $[2, 15]$ for all datasets, with optimal values of 6, 6, 7, and 12 for NYCBike, NYCTaxi, PEMSD4, and PEMSD8, respectively. The batch size is set to 32. The hidden dimension is based on the original implementations in the open-source code of each base model. For additional details, please refer to our public code repository: https://github.com/bigscity/TGDL.

B. Overall Performance

1) Experiments With Different Base-Models: The experimental results for the "experiments with different base-models" scenario are presented in Table II. Here, the TGDL framework combined with each base model is denoted as "base model+". We use three common metrics to evaluate prediction accuracy: Mean Average Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE). Each experiment was run five times with different random seeds, and the average performance along with standard deviation is reported as "$\text{mean}_{\pm\text{sd}}$". The relative error reduction rate (RERR) of TGDL

TABLE II
PERFORMANCE COMPARISON ON ALL FOUR DATASETS REGARDING MAE, RMSE, AND MAPE (%). THE BEST RESULT IS BOLD, WHILE THE SECOND-BEST RESULT IS UNDERLINED. "IN" AND "OUT" DENOTE INFLOW AND OUTFLOW. "30 MIN" AND "2 HOURS" DENOTE THE PREDICTION HORIZON OF 30 MINUTES (1 STEP) AND 2 HOURS (4 STEPS). "RERR" IS THE RELATIVE ERROR REDUCTION RATE OF TGDL ENHANCED MODELS COMPARED WITH THE ORIGINAL BASE MODELS.

Dataset	NYCBike						NYCTaxi					
Metric	MAE ↓		RMSE ↓		MAPE ↓		MAE ↓		RMSE ↓		MAPE ↓	
Type	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out
STGCN	$5.26_{\pm0.02}$	$5.53_{\pm0.04}$	$7.65_{\pm0.05}$	$8.23_{\pm0.11}$	$22.58_{\pm0.29}$	$23.13_{\pm0.38}$	$12.87_{\pm0.21}$	$10.93_{\pm0.13}$	$23.09_{\pm0.50}$	$23.62_{\pm0.47}$	$17.25_{\pm0.16}$	$17.23_{\pm0.20}$
STGCN+	$\underline{5.07}_{\pm0.05}$	$5.36_{\pm0.02}$	$\underline{7.36}_{\pm0.11}$	$\underline{7.95}_{\pm0.07}$	$\underline{21.77}_{\pm0.08}$	$\underline{22.55}_{\pm0.05}$	$11.84_{\pm0.12}$	$\underline{9.68}_{\pm0.15}$	$21.17_{\pm0.26}$	$17.61_{\pm0.79}$	$17.13_{\pm0.84}$	$\underline{16.80}_{\pm0.01}$
RERR	-3.61%	-3.07%	-3.79%	-3.40%	-3.59%	-2.51%	-8.00%	-11.44%	-8.32%	-25.45%	-0.70%	-2.50%
GWNet	$5.15_{\pm0.02}$	$5.43_{\pm0.02}$	$7.38_{\pm0.03}$	$8.01_{\pm0.04}$	$22.67_{\pm0.21}$	$23.24_{\pm0.20}$	$12.70_{\pm0.02}$	$10.39_{\pm0.08}$	$22.48_{\pm0.06}$	$18.29_{\pm0.12}$	$17.45_{\pm1.12}$	$17.79_{\pm0.48}$
GWNet+	$\mathbf{4.97}_{\pm0.02}$	$\mathbf{5.25}_{\pm0.01}$	$\mathbf{7.12}_{\pm0.04}$	$\mathbf{7.75}_{\pm0.05}$	$\mathbf{21.61}_{\pm0.21}$	$\mathbf{22.28}_{\pm0.02}$	$\mathbf{11.53}_{\pm0.08}$	$\mathbf{9.32}_{\pm0.04}$	$\underline{20.97}_{\pm0.21}$	$\mathbf{16.15}_{\pm0.07}$	$\mathbf{15.78}_{\pm0.21}$	$\mathbf{16.28}_{\pm0.32}$
RERR	-3.50%	-3.31%	-3.52%	-3.25%	-4.68%	-4.13%	-9.21%	-10.30%	-6.72%	-11.70%	-9.57%	-8.49%
MTGNN	$5.26_{\pm0.03}$	$5.57_{\pm0.03}$	$7.68_{\pm0.06}$	$8.31_{\pm0.03}$	$22.77_{\pm0.10}$	$23.83_{\pm0.61}$	$12.71_{\pm0.03}$	$10.19_{\pm0.03}$	$22.37_{\pm0.05}$	$18.00_{\pm0.12}$	$17.03_{\pm0.06}$	$17.71_{\pm0.25}$
MTGNN+	$5.12_{\pm0.01}$	$5.42_{\pm0.01}$	$7.46_{\pm0.04}$	$8.12_{\pm0.02}$	$22.28_{\pm0.13}$	$23.18_{\pm0.03}$	$11.94_{\pm0.15}$	$9.71_{\pm0.09}$	$21.07_{\pm0.26}$	$\underline{16.90}_{\pm0.19}$	$16.38_{\pm0.36}$	$16.98_{\pm0.47}$
RERR	-2.66%	-2.69%	-2.86%	-2.29%	-2.15%	-2.73%	-6.06%	-4.71%	-5.81%	-6.11%	-3.82%	-4.12%
MSDR	$5.65_{\pm0.10}$	$6.00_{\pm0.14}$	$8.26_{\pm0.15}$	$8.99_{\pm0.23}$	$24.54_{\pm0.59}$	$25.89_{\pm0.10}$	$18.49_{\pm1.20}$	$15.89_{\pm1.02}$	$33.7_{\pm2.09}$	$33.17_{\pm1.67}$	$25.27_{\pm1.73}$	$25.78_{\pm1.93}$
MSDR+	$5.25_{\pm0.05}$	$5.55_{\pm0.07}$	$7.68_{\pm0.11}$	$8.31_{\pm0.10}$	$23.22_{\pm0.24}$	$23.86_{\pm0.30}$	$14.72_{\pm0.10}$	$11.53_{\pm0.28}$	$26.31_{\pm0.34}$	$21.44_{\pm0.63}$	$20.61_{\pm1.00}$	$20.37_{\pm0.64}$
RERR	-7.08%	-7.50%	-7.02%	-7.56%	-5.38%	-7.84%	-20.39%	-27.44%	-21.93%	-35.36%	-18.44%	-20.99%
STSSL	$5.18_{\pm0.04}$	$5.44_{\pm0.05}$	$7.56_{\pm0.06}$	$8.15_{\pm0.03}$	$22.53_{\pm0.17}$	$23.22_{\pm0.08}$	$12.19_{\pm0.14}$	$10.00_{\pm0.09}$	$21.70_{\pm0.18}$	$19.05_{\pm0.37}$	$16.51_{\pm0.04}$	$17.21_{\pm0.06}$
STSSL+	$5.10_{\pm0.02}$	$\underline{5.35}_{\pm0.03}$	$7.45_{\pm0.04}$	$8.03_{\pm0.03}$	$22.36_{\pm0.04}$	$22.99_{\pm0.05}$	$\underline{11.83}_{\pm0.10}$	$9.74_{\pm0.04}$	$\mathbf{20.86}_{\pm0.10}$	$17.06_{\pm0.13}$	$\underline{16.36}_{\pm0.01}$	$17.06_{\pm0.02}$
RERR	-1.54%	-1.65%	-1.46%	-1.47%	-0.76%	-0.99%	-2.95%	-2.60%	-3.87%	-10.45%	-0.91%	-0.87%

Dataset	PEMSD4						PEMSD8					
Metric	MAE ↓		RMSE ↓		MAPE ↓		MAE ↓		RMSE ↓		MAPE ↓	
Type	30 min	2 hours	30 min	2 hours	30 min	2 hours	30 min	2 hours	30 min	2 hours	30 min	2 hours
STGCN	$28.88_{\pm0.55}$	$30.07_{\pm0.14}$	$42.36_{\pm0.89}$	$44.20_{\pm0.21}$	$22.21_{\pm0.71}$	$20.96_{\pm0.45}$	$23.61_{\pm0.42}$	$26.96_{\pm0.34}$	$34.69_{\pm0.64}$	$40.55_{\pm0.73}$	$14.65_{\pm0.30}$	$15.35_{\pm0.49}$
STGCN+	$12.45_{\pm0.20}$	$15.87_{\pm0.03}$	$20.00_{\pm0.21}$	$26.37_{\pm0.27}$	$8.11_{\pm0.16}$	$9.92_{\pm0.11}$	$9.75_{\pm0.02}$	$12.45_{\pm0.21}$	$15.97_{\pm0.15}$	$21.46_{\pm0.31}$	$5.68_{\pm0.07}$	$6.98_{\pm0.27}$
RERR	-56.89%	-47.22%	-52.79%	-40.34%	-63.48%	-52.67%	-58.70%	-53.82%	-53.96%	-47.08%	-61.23%	-54.53%
GWNet	$12.52_{\pm0.09}$	$18.39_{\pm0.15}$	$19.96_{\pm0.07}$	$29.18_{\pm0.36}$	$8.40_{\pm0.34}$	$11.80_{\pm0.25}$	$9.83_{\pm0.09}$	$14.59_{\pm0.16}$	$16.20_{\pm0.10}$	$23.46_{\pm0.19}$	$5.92_{\pm0.16}$	$8.48_{\pm0.19}$
GWNet+	$\mathbf{11.22}_{\pm0.05}$	$\mathbf{14.58}_{\pm0.03}$	$\mathbf{18.27}_{\pm0.05}$	$\mathbf{24.61}_{\pm0.25}$	$\underline{7.34}_{\pm0.02}$	$\underline{9.37}_{\pm0.08}$	$\underline{8.62}_{\pm0.04}$	$\mathbf{11.36}_{\pm0.07}$	$\mathbf{14.68}_{\pm0.04}$	$\mathbf{20.30}_{\pm0.14}$	$\mathbf{4.83}_{\pm0.06}$	$\mathbf{6.52}_{\pm0.04}$
RERR	-10.38%	-20.72%	-8.47%	-15.66%	-8.82%	-20.59%	-12.31%	-22.14%	-9.38%	-13.47%	-22.25%	-23.11%
MTGNN	$12.28_{\pm0.07}$	$18.38_{\pm0.15}$	$19.84_{\pm0.09}$	$30.40_{\pm0.24}$	$7.70_{\pm0.10}$	$11.23_{\pm0.32}$	$9.65_{\pm0.08}$	$13.55_{\pm0.19}$	$15.95_{\pm0.09}$	$23.11_{\pm0.18}$	$5.88_{\pm0.11}$	$7.78_{\pm0.30}$
MTGNN+	$\underline{11.29}_{\pm0.04}$	$\underline{15.39}_{\pm0.07}$	$\underline{18.62}_{\pm0.03}$	$\underline{25.84}_{\pm0.25}$	$\mathbf{6.88}_{\pm0.03}$	$\mathbf{9.30}_{\pm0.13}$	$\mathbf{8.60}_{\pm0.02}$	$\underline{11.43}_{\pm0.11}$	$\underline{14.82}_{\pm0.05}$	$\underline{20.31}_{\pm0.18}$	$\underline{4.95}_{\pm0.09}$	$\underline{6.89}_{\pm0.08}$
RERR	-8.06%	-16.27%	-6.15%	-15.00%	-10.65%	-17.19%	-10.88%	-15.65%	-7.08%	-12.12%	-18.79%	-11.44%
MSDR	$27.75_{\pm2.10}$	$25.66_{\pm1.70}$	$44.44_{\pm1.40}$	$38.81_{\pm1.96}$	$19.01_{\pm0.94}$	$18.25_{\pm1.92}$	$23.21_{\pm0.36}$	$23.56_{\pm0.22}$	$39.46_{\pm0.51}$	$34.75_{\pm0.38}$	$14.03_{\pm0.33}$	$14.69_{\pm0.38}$
MSDR+	$19.09_{\pm0.56}$	$20.59_{\pm0.68}$	$31.27_{\pm0.91}$	$33.17_{\pm1.22}$	$12.96_{\pm0.26}$	$14.15_{\pm0.54}$	$17.65_{\pm0.14}$	$17.98_{\pm0.21}$	$34.72_{\pm0.18}$	$30.62_{\pm0.23}$	$9.91_{\pm0.36}$	$10.42_{\pm0.25}$
RERR	-31.21%	-19.76%	-29.64%	-14.53%	-31.83%	-22.47%	-23.96%	-23.68%	-12.01%	-11.88%	-29.37%	-29.07%
STSSL	$28.87_{\pm0.64}$	$29.22_{\pm0.57}$	$41.85_{\pm0.66}$	$43.62_{\pm0.65}$	$21.48_{\pm0.49}$	$19.90_{\pm0.59}$	$22.95_{\pm0.25}$	$27.18_{\pm0.85}$	$34.26_{\pm0.28}$	$40.01_{\pm0.81}$	$14.74_{\pm0.22}$	$16.03_{\pm0.46}$
STSSL+	$12.90_{\pm0.21}$	$16.17_{\pm0.55}$	$20.51_{\pm0.20}$	$26.81_{\pm0.82}$	$8.22_{\pm0.17}$	$10.11_{\pm0.34}$	$10.50_{\pm0.06}$	$13.77_{\pm0.21}$	$16.89_{\pm0.16}$	$22.91_{\pm0.23}$	$6.30_{\pm0.05}$	$8.40_{\pm0.15}$
RERR	-55.32%	-44.66%	-50.99%	-38.54%	-61.73%	-49.20%	-54.25%	-49.34%	-50.70%	-42.74%	-57.26%	-47.60%

compared to the original base models is also shown in Table II. From the table, we make the following observations.

• Firstly, for all base models, the performance of TGDL +base-models is significantly better than the corresponding original base models in every case, with an average error reduction rate of 19.37%, 17.62%, and 19.49% in MAE, RMSE, and MAPE, respectively. This validates the effectiveness of TGDL as a framework to enhance the predictive capabilities of various GNN-based models. Such effectiveness can also be generalized to more traffic data types and even other spatiotemporal domains beyond traffic. (see Sects. II.D and II.E of the Supplementary Materials).

• Secondly, GWNet+ outperforms other enhanced models for two main reasons: (1) Its base version, GWNet, is already a powerful model capable of capturing complex spatiotemporal dependencies, thereby providing a strong foundation for accurate prediction. This results in a lower model-induced (MI) error. (2) With the integration of TGDL, which reduces data-induced (DI) error, GWNet's modeling capabilities are fully leveraged. Consequently, GWNet+ achieves the lowest total error, which is defined as the summation of the MI and DI errors.

• Thirdly, although the enhanced GWNet achieves the best overall performance, the relative improvement brought by TGDL is more pronounced in simpler models such as STGCN. This finding demonstrates that TGDL effectively reduces the intrinsic difficulty of the prediction task, thereby diminishing the necessity for highly complex models. As a result, TGDL enables lightweight models to achieve competitive accuracy, making them more suitable for practical applications where computational resources may be limited.

• Fourthly, for base models with relatively lower original performance, integrating the TGDL framework results in a greater enhancement, reducing performance differences between base models with TGDL. Fig. 4(a)–(c) provide additional insight into this phenomenon. In this figure, we plot the standard deviations of prediction performance for TGDL +base models and the original base models. The white bars show the standard deviations of base model performance, while the blue bars represent those

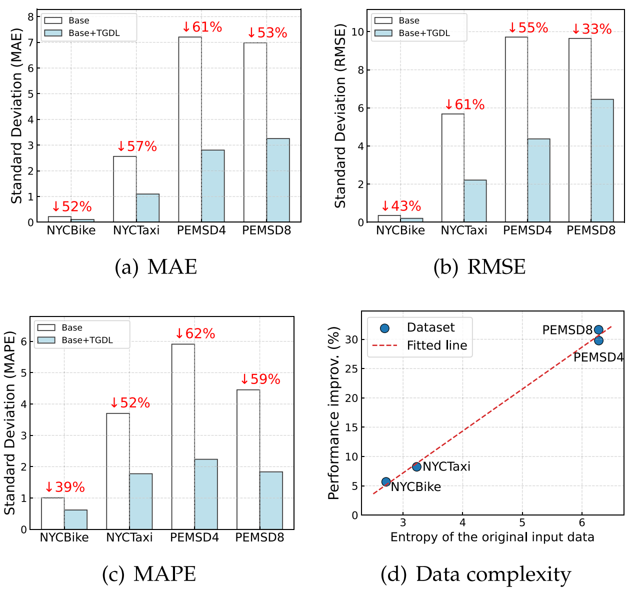


Fig. 4. (a)–(c): Performance gap before and after TGDL enhancement. The percentage after "↓" means the relative reduction rate of performance gaps. (d) Impact of data complexity on performance improvement introduced by our proposed TGDL framework.

of TGDL +base models. The reduced performance standard deviations for TGDL +base models indicate that, with decomposition by TGDL, the complexity of the subproblem decreases, narrowing the performance gap across different models.

• Finally, the performance gains for the PEMS datasets are significantly greater than those for the NYC datasets because datasets with higher uncertainty, measured by the entropy $\mathrm{H}(Y|X)$, tend to benefit more from our TGDL. Fig. 4(d) illustrates the relationship between TGDL's average performance gains and data complexity for each dataset, where each point represents a dataset, and the fitted curve highlights the trend. As shown in Fig. 4(d), datasets like PEMSD4 and PEMSD8, which have higher entropy, achieve larger improvements than NYCBike and NYCTaxi. This disparity arises from dataset composition: the NYC datasets, limited to a single vehicle type within one city, exhibit simpler traffic patterns, while the PEMS datasets include multiple vehicle types across diverse regions, resulting in more complex and heterogeneous patterns. As a result, TGDL achieves greater performance improvements on the PEMS datasets due to their richer structure and decomposability. Therefore, calculating the entropy of a dataset before applying TGDL can help identify where the model is likely to provide the most significant benefits.

2) Comparison With Decomposition Approaches: Table III presents the results of TGDL in comparison to other decomposition-enhanced baselines. Here, the base model for TGDL is STGCN, chosen as the most widely cited base model. The following observations can be made:

• Firstly, the classic non-deep methods, HA and SVR, yield suboptimal results due to their limited expressive power. In contrast, deep learning models, particularly those enhanced with TGDL, can capture complex spatiotemporal relationships, resulting in more accurate predictions.

• Secondly, the two-stage decomposition prediction approaches generally underperform compared to end-to-end paradigms, primarily due to the objective inconsistency between the decomposition and prediction stages.

• Thirdly, among the baselines with signal decomposition, SCNN achieves the best average performance. Compared to other signal decomposition baselines, SCNN decomposes the input data into a wider variety of component types, increasing the likelihood of their independence from each other. This approach aligns with the CI principle, benefiting performance. However, none of these baselines include explicit mechanisms to ensure maximal independence of decomposition components, leading to lower performance than TGDL and its variants.

• Fourthly, the variants TGDL-METIS and TGDL-DBH outperform other representative baselines, highlighting that graph structure decomposition, in addition to signal decomposition, is also crucial for ensuring component independence and reducing prediction error. Furthermore, the TGDL framework in these variants promotes component independence, which contributes to improved performance.

• Finally, our proposed TGDL consistently outperforms all other baselines across all four datasets (including more recent SOTA methods in Sect. II.C of the Supplementary Materials), verifying the effectiveness of our method. The performance of TGDL-METIS and TGDL-DBH is slightly lower than that of TGDL, likely because these variants separately optimize the structure and signal decomposition modules, whereas TGDL jointly optimizes both modules in an end-to-end manner.

C. Further Analysis of TGDL

1) Ablation Study: We conduct an ablation study to analyze how each of the proposed components affects the final prediction performance. We prepare five variants for comparison: 1) *w/o GSD:* This variant removes the graph structure decomposition module, using the original graph as the input structure for each ST block. 2) *w/o RDF:* This variant removes the residual decomposition flow described in Section IV-C. 3) *w/o IR:* This variant excludes all independence regularization terms, including those in (25) and (36). 4) *w/o GSD+IR:* This variant is the combination of *w/o GSD* and *w/o IR*. 5) *w/o RDF+IR:* This variant is the combination of *w/o RDF* and *w/o IR*. The last two variants aim to examine the synergistic effects among different modules and constraints.

The results, using STGCN as the base model, are shown in Table IV. As illustrated, each component contributes to performance improvement. Specifically, the performance of variants (1) and (2) shows a significant decrease on the PEMSD datasets, suggesting that our proposed decomposition of structure and signal is particularly effective for modeling traffic data with complex component coupling. The performance drop in variant (3) highlights the effectiveness of the independence constraint. Moreover, variants (4) and (5) exhibit a notable decline in performance, suggesting the presence of a synergistic effect among the proposed modules, i.e., the combined effect of multiple modules exceeds the sum of their individual contributions. Additionally, variant (5) performs worse than variant (4), indicating that the

TABLE III
PERFORMANCE COMPARISON OF VARIOUS METHODS REGARDING MAE, RMSE, AND MAPE (%), WITH THE BEST IN BOLD AND THE SECOND-BEST UNDERLINED

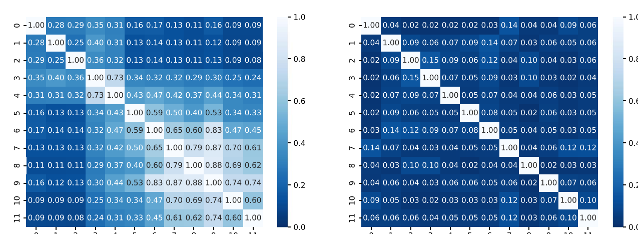
Dataset	**NYCBike**						**NYCTaxi**					
Metric	MAE ↓		RMSE ↓		MAPE ↓		MAE ↓		RMSE ↓		MAPE ↓	
Type	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out
HA	10.85	11.09	16.91	17.50	45.68	46.98	36.76	28.51	66.87	56.68	46.72	44.02
SVR	8.48	8.59	13.67	14.31	34.63	34.63	35.81	28.65	68.91	58.05	45.62	43.47
PTD-XFMR	7.77$_{\pm0.08}$	8.12$_{\pm0.12}$	11.21$_{\pm0.13}$	12.44$_{\pm0.22}$	33.43$_{\pm0.28}$	35.12$_{\pm0.43}$	17.53$_{\pm0.15}$	13.56$_{\pm0.06}$	30.89$_{\pm0.32}$	23.86$_{\pm0.14}$	22.79$_{\pm0.15}$	22.06$_{\pm0.08}$
EMD-XFMR	7.18$_{\pm0.21}$	7.83$_{\pm0.24}$	10.61$_{\pm0.36}$	11.57$_{\pm0.44}$	31.35$_{\pm0.76}$	33.73$_{\pm0.74}$	17.15$_{\pm0.35}$	12.62$_{\pm0.13}$	29.19$_{\pm0.61}$	22.32$_{\pm0.28}$	21.32$_{\pm0.38}$	21.38$_{\pm0.21}$
CoST	6.58$_{\pm0.01}$	7.06$_{\pm0.03}$	9.47$_{\pm0.03}$	10.29$_{\pm0.04}$	30.10$_{\pm0.23}$	31.55$_{\pm0.38}$	17.45$_{\pm0.16}$	13.73$_{\pm0.05}$	30.27$_{\pm0.29}$	24.96$_{\pm0.28}$	24.70$_{\pm0.22}$	23.66$_{\pm0.17}$
D2STGNN	5.63$_{\pm0.34}$	5.95$_{\pm0.45}$	8.07$_{\pm0.57}$	8.80$_{\pm0.95}$	24.74$_{\pm1.56}$	25.73$_{\pm1.45}$	16.50$_{\pm0.99}$	13.60$_{\pm0.42}$	29.24$_{\pm1.64}$	25.29$_{\pm0.67}$	20.07$_{\pm0.55}$	19.20$_{\pm0.22}$
STWave	7.11$_{\pm0.36}$	7.63$_{\pm0.47}$	10.30$_{\pm0.53}$	11.33$_{\pm0.81}$	30.39$_{\pm1.34}$	32.60$_{\pm1.79}$	16.38$_{\pm0.54}$	12.55$_{\pm0.25}$	28.89$_{\pm1.16}$	22.64$_{\pm0.51}$	21.31$_{\pm0.66}$	20.78$_{\pm0.40}$
SCNN	6.10$_{\pm0.14}$	6.59$_{\pm0.14}$	8.79$_{\pm0.25}$	9.87$_{\pm0.26}$	27.77$_{\pm0.60}$	29.31$_{\pm0.61}$	14.32$_{\pm0.30}$	11.39$_{\pm0.16}$	24.67$_{\pm0.59}$	19.63$_{\pm0.31}$	21.03$_{\pm0.34}$	21.23$_{\pm0.23}$
TGDL-METIS	5.18$_{\pm0.02}$	<u>5.46</u>$_{\pm0.02}$	7.62$_{\pm0.04}$	<u>8.09</u>$_{\pm0.07}$	22.11$_{\pm0.12}$	<u>23.09</u>$_{\pm0.34}$	<u>11.94</u>$_{\pm0.10}$	<u>9.83</u>$_{\pm0.15}$	<u>21.62</u>$_{\pm0.22}$	18.72$_{\pm0.52}$	18.81$_{\pm0.55}$	<u>17.25</u>$_{\pm0.30}$
TGDL-DBH	<u>5.16</u>$_{\pm0.04}$	5.50$_{\pm0.06}$	<u>7.50</u>$_{\pm0.05}$	8.15$_{\pm0.32}$	<u>22.10</u>$_{\pm0.52}$	<u>23.09</u>$_{\pm0.62}$	12.35$_{\pm0.03}$	10.01$_{\pm0.15}$	21.81$_{\pm0.22}$	<u>18.30</u>$_{\pm0.27}$	<u>18.20</u>$_{\pm0.05}$	18.54$_{\pm0.32}$
TGDL	**5.07**$_{\pm0.05}$	**5.36**$_{\pm0.02}$	**7.36**$_{\pm0.11}$	**7.95**$_{\pm0.07}$	**21.77**$_{\pm0.08}$	**22.55**$_{\pm0.05}$	**11.84**$_{\pm0.12}$	**9.68**$_{\pm0.15}$	**21.17**$_{\pm0.26}$	**17.61**$_{\pm0.79}$	**17.13**$_{\pm0.84}$	**16.80**$_{\pm0.01}$
Dataset	**PEMSD4**						**PEMSD8**					
Metric	MAE ↓		RMSE ↓		MAPE ↓		MAE ↓		RMSE ↓		MAPE ↓	
Type	30 min	2 hours	30 min	2 hours	30 min	2 hours	30 min	2 hours	30 min	2 hours	30 min	2 hours
HA	40.70	63.33	61.78	89.72	32.80	53.44	37.03	55.74	54.15	77.98	24.84	36.67
SVR	40.80	47.59	61.57	71.09	32.75	34.17	36.83	41.43	54.88	59.99	26.20	26.67
PTD-XFMR	20.19$_{\pm0.32}$	28.18$_{\pm0.18}$	30.19$_{\pm0.24}$	42.24$_{\pm0.45}$	12.25$_{\pm0.28}$	18.37$_{\pm0.25}$	18.22$_{\pm0.24}$	28.52$_{\pm0.11}$	26.92$_{\pm0.41}$	41.67$_{\pm0.14}$	10.17$_{\pm0.21}$	16.42$_{\pm0.20}$
EMD-XFMR	19.27$_{\pm0.63}$	26.95$_{\pm0.51}$	28.55$_{\pm0.73}$	39.81$_{\pm0.75}$	11.76$_{\pm0.54}$	17.07$_{\pm0.52}$	17.75$_{\pm0.45}$	27.02$_{\pm0.39}$	25.18$_{\pm0.63}$	41.37$_{\pm0.62}$	9.86$_{\pm0.45}$	15.58$_{\pm0.41}$
CoST	21.00$_{\pm0.46}$	24.59$_{\pm0.60}$	32.71$_{\pm0.56}$	37.07$_{\pm0.73}$	14.79$_{\pm0.39}$	17.96$_{\pm0.71}$	20.87$_{\pm0.03}$	25.04$_{\pm0.07}$	32.78$_{\pm0.11}$	37.90$_{\pm0.13}$	12.00$_{\pm0.05}$	14.78$_{\pm0.06}$
D2STGNN	17.07$_{\pm1.99}$	22.91$_{\pm1.26}$	24.64$_{\pm1.35}$	34.78$_{\pm1.19}$	15.21$_{\pm5.76}$	16.65$_{\pm3.07}$	15.51$_{\pm0.18}$	23.93$_{\pm0.83}$	23.25$_{\pm0.05}$	34.98$_{\pm0.57}$	12.37$_{\pm0.62}$	21.85$_{\pm1.44}$
STWave	18.46$_{\pm1.12}$	26.03$_{\pm1.51}$	28.03$_{\pm1.45}$	39.79$_{\pm1.76}$	11.51$_{\pm0.96}$	16.89$_{\pm1.14}$	16.92$_{\pm1.59}$	26.20$_{\pm0.59}$	25.09$_{\pm2.04}$	39.45$_{\pm1.23}$	9.62$_{\pm0.87}$	15.46$_{\pm0.78}$
SCNN	13.79$_{\pm0.28}$	21.72$_{\pm0.69}$	22.07$_{\pm0.45}$	34.83$_{\pm1.19}$	8.37$_{\pm0.19}$	13.74$_{\pm0.44}$	10.78$_{\pm0.26}$	16.82$_{\pm0.18}$	17.19$_{\pm0.40}$	26.20$_{\pm0.31}$	6.24$_{\pm0.15}$	9.91$_{\pm0.09}$
TGDL-METIS	13.21$_{\pm0.45}$	17.15$_{\pm0.21}$	21.30$_{\pm0.76}$	28.80$_{\pm0.44}$	8.33$_{\pm0.39}$	10.36$_{\pm0.11}$	10.85$_{\pm0.08}$	13.29$_{\pm0.22}$	17.45$_{\pm0.29}$	22.40$_{\pm0.12}$	6.46$_{\pm0.15}$	7.64$_{\pm0.10}$
TGDL-DBH	<u>12.82</u>$_{\pm0.55}$	<u>16.35</u>$_{\pm0.27}$	<u>21.07</u>$_{\pm0.13}$	<u>27.30</u>$_{\pm0.27}$	<u>8.27</u>$_{\pm0.18}$	<u>10.09</u>$_{\pm0.15}$	<u>10.31</u>$_{\pm0.04}$	<u>13.18</u>$_{\pm0.12}$	<u>16.59</u>$_{\pm0.22}$	<u>22.20</u>$_{\pm0.62}$	<u>6.19</u>$_{\pm0.10}$	<u>7.60</u>$_{\pm0.32}$
TGDL	**12.45**$_{\pm0.20}$	**15.87**$_{\pm0.03}$	**20.00**$_{\pm0.21}$	**26.37**$_{\pm0.27}$	**8.11**$_{\pm0.16}$	**9.92**$_{\pm0.11}$	**9.75**$_{\pm0.02}$	**12.45**$_{\pm0.21}$	**15.97**$_{\pm0.15}$	**21.46**$_{\pm0.31}$	**5.68**$_{\pm0.07}$	**6.98**$_{\pm0.27}$

TABLE IV
ABLATION STUDY ON ALL DATASETS REGARDING THE OVERALL PERFORMANCE

Dataset	**NYCBike**			**NYCTaxi**		
Metric	MAE	RMSE	MAPE	MAE	RMSE	MAPE
(0): TGDL	**5.22**	**7.66**	**22.52**	**10.63**	**18.88**	**17.07**
(1): w/o GSD	5.28	<u>7.73</u>	<u>22.54</u>	<u>10.71</u>	<u>19.25</u>	17.36
(2): w/o RDF	5.30	7.79	22.78	11.37	21.01	17.43
(3): w/o IR	<u>5.27</u>	7.75	22.57	11.40	21.34	<u>17.29</u>
(4): w/o GSD+IR	5.45	8.05	23.14	11.94	21.76	17.61
(5): w/o RDF+IR	5.89	8.46	25.77	12.65	23.13	19.30
Dataset	**PEMSD4**			**PEMSD8**		
Metric	MAE	RMSE	MAPE	MAE	RMSE	MAPE
(0): TGDL	**14.25**	**23.51**	**8.96**	**11.56**	**19.46**	**6.57**
(1): w/o GSD	15.63	25.33	9.99	12.67	21.40	7.24
(2): w/o RDF	<u>15.35</u>	<u>25.08</u>	9.82	13.13	21.67	7.49
(3): w/o IR	15.67	25.58	<u>9.80</u>	<u>12.63</u>	<u>21.13</u>	<u>7.22</u>
(4): w/o GSD+IR	16.12	25.97	10.24	13.09	22.44	7.28
(5): w/o RDF+IR	16.79	26.93	10.32	14.32	24.78	7.96

synergy between signal decomposition and the independence regularization term plays a particularly critical role in enhancing model performance.

2) Analysis of Independence Regularization: Guided by the CI principle, the TGDL framework incorporates two independence regularizations to encourage mutual independence among the decomposed components across different subgraphs. Fig. 5 illustrates the effects of independence regularization using the PEMSD8 dataset with STGCN-based TGDL as an example. In this experiment, we decompose the traffic state of the PEMSD8 dataset into twelve subgraphs. The correlations between the decomposed components of different subgraphs are shown in a correlation matrix, where the (k, k')-th element represents the correlation between the k-th and k'-th subgraphs. Values closer to 1 indicate higher correlation.



(a) w/o ind (b) TGDL

Fig. 5. Visualization of correlation matrices among predictions of different components on the PEMSD8 dataset. Higher values and brighter colors signify lower levels of independence.

Fig. 5(a) presents the correlation matrix for TGDL without independence regularization (denoted as *w/o ind*), while Fig. 5(b) shows the matrix for the complete TGDL model. As illustrated, the correlations among subgraphs in TGDL are substantially lower than those in *w/o ind*, demonstrating that the proposed independence regularization effectively promotes component independence. Notably, some non-zero inter-component correlations persist after decomposition, indicating that TGDL retains the flexibility to model long-range or global interactions when necessary. Additionally, Table V reports results for other datasets, summarizing the average correlations from the upper triangular portion of the correlation matrix. As shown, independence regularization reduces correlation by over 50%, thereby significantly lowering DI errors.

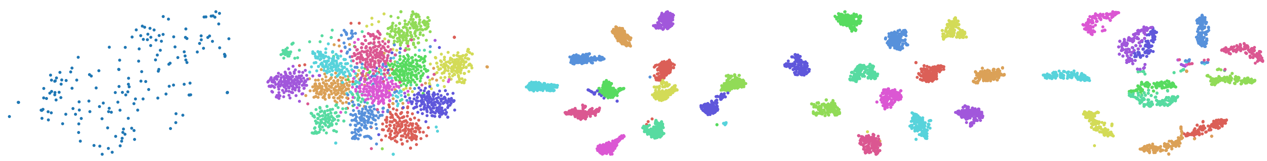
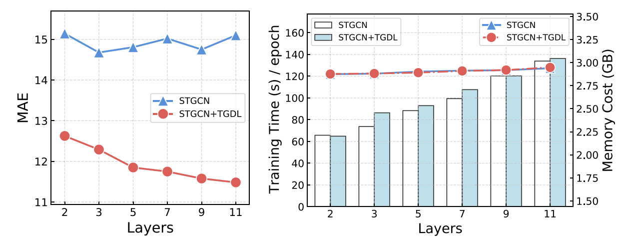

(a) w/o TGDL (b) w/o training ($S = 0.27$) (c) w/o ind ($S = 0.68$) (d) TGDL ($S = 0.77$) (e) SCNN ($S = 0.50$)

Fig. 6. Scatter plots for embeddings trained by different models. The color indicates distinct decomposed components. "S" is Silhouette Score.

TABLE V
NORMALIZED MUTUAL INFORMATION BETWEEN PREDICTIONS OF DIFFERENT COMPONENTS. Δ MEASURES THE RELATIVE REDUCTION RATE

Dataset	NYCBike	NYCTaxi	PEMSD4	PEMSD8
w/o ind	0.2305	0.6197	0.2306	0.3616
TGDL	0.0957	0.1678	0.0603	0.0585
Δ	-58.48%	-72.92%	-73.85%	-83.82%



(a) Performance (b) Time/Memory Cost

Fig. 7. Scalability vs. model complexity.

3) Analysis of Signal Representations: To further examine the role of the independence regularization terms in TGDL, we visualize the embedding distribution for the signal representation matrix $\boldsymbol{H}_t^{(k)}$ generated by the signal decomposition in (32). Here, $\boldsymbol{H}_t^{(k)}$ represents the t-th sample of subgraph k. For the visualization, we generate $\boldsymbol{H}_t^{(k)}$ for all samples in the PEMSD8 dataset and apply the t-SNE algorithm [91] to convert $\boldsymbol{H}_t^{(k)}$ into two-dimensional embedding vectors.

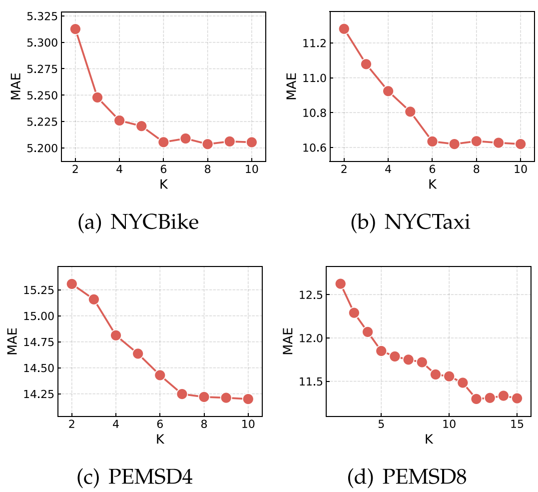
Fig. 6 illustrates the embedding distributions under various conditions. Specifically, Fig. 6(a) shows the embeddings of the base model (STGCN), Fig. 6(b) presents the TGDL model without training, Fig. 6(c) depicts the TGDL model without independence regularization, and Fig. 6(d) represents the complete TGDL model. For comparison, Fig. 6(e) shows the embedding distribution of the best signal decomposition-based baseline, SCNN. In these figures, each scatter point represents a sample, with colors distinguishing different subgraphs. To quantify these distributions, we calculate the silhouette score [92], which reflects the cohesion and separation within clusters.

The embeddings generated by the base model are relatively dispersed, lacking clear clustering patterns. In TGDL without training, the embeddings show some spread, which is expected given that training has not started. For TGDL without independence regularization, most embeddings are well separated, though some overlap remains. The complete TGDL model, however, clearly separates all embeddings, demonstrating the effectiveness of the independence regularization. We also find that the decomposed components in Fig. 6(d) correspond to distinct and interpretable traffic patterns (see Sect. I.A of the Supplementary Materials). Furthermore, while the SCNN model can distinguish the embeddings, it struggles to cluster samples from the same subcomponent effectively. Comparing the silhouette scores between SCNN and the TGDL variants, we find that the decomposition effect of TGDL without independence regularization is still superior to SCNN. This result helps explain TGDL's performance advantage over SCNN.

4) Scalability Experiments: The TGDL model decomposes the input traffic data into multiple components and employs multiple base models to predict future traffic states. As a result, a TGDL model with K base models contains K times the parameters of a single base model. This raises a natural question: is the observed performance improvement of TGDL merely a consequence of increased model parameters? To investigate this, we conduct scalability experiments, as shown in Fig. 7. In these experiments, we evaluate the performance, training time, and memory usage of TGDL with 2 to 11 subgraphs, using STGCN as the base model and PEMSD8 as the dataset. For comparison, we construct standard STGCN models with 2 to 11 layers, connected via residual connections, so that an STGCN model with K layers has approximately the same parameter count as TGDL with K STGCN base models.

As shown in Fig. 7, increasing the number of layers allows the TGDL-enhanced model to effectively utilize additional parameters for improved performance. In contrast, the performance of the stacked base model remains stable, indicating that STGCN may have already minimized MI errors for the original data. However, TGDL can further reduce the total error by decreasing the DI error in each subgraph. Regarding training time and memory usage, both models perform similarly, with only a slight increase in training time as parameter count grows and memory usage remaining relatively stable between 2–3 GB. Overall, TGDL enhances the scalability of the base model without increasing time or memory costs.

Additionally, we compare our model with recent unified large models in terms of generalization performance and training cost (see Sect. II.B of the Supplementary Materials). The results indicate that, while unified models provide some cross-scenario generalization, they require large parameter sizes and still need fine-tuning for new scenarios. In contrast, TGDL achieves comparable or superior performance with a lightweight base model,



(a) NYCBike (b) NYCTaxi

(c) PEMSD4 (d) PEMSD8

Fig. 8. Impact of the number of subgraphs K on MAE.

demonstrating greater efficiency and adaptability at significantly lower computational cost.

5) Impact of Hyper-Parameters: In our experiments, we use the default hyper-parameter setups from the official open-source code of the base models. These parameters have been carefully configured by the original authors to ensure state-of-the-art performance. The only hyperparameter introduced by our approach is the number K of decomposed subgraphs. To examine the impact of this parameter, we vary K from 2 to 15 across all four datasets, using STGCN as the base model. As shown in Fig. 8, too few subgraphs significantly degrade performance, as this configuration fails to capture the effects of multiple factors within the traffic data. As K increases, performance stabilizes or improves marginally. Based on these findings, we set K to 6, 6, 7, and 12 for NYCBike, NYCTaxi, PEMSD4, and PEMSD8, respectively, achieving an optimal balance between performance and model complexity.

VI. Conclusion and Future Work

Conclusion: This work addressed the traffic prediction problem from a decomposition perspective. Specifically, we adopted a "decompose, then predict" paradigm and established a robust theoretical foundation for it using information theory. We derived sufficient conditions under which a decomposition algorithm could improve traffic prediction, formalized as the Component Independence Principle. Building on this, we proposed a flexible Theory-guided Graph Decomposition Learning (TGDL) framework, which comprises (1) a graph structure decomposition module with structure independence regularization that divides the input traffic graph into independent subgraphs, (2) a graph signal decomposition module that iteratively decouples signals for each subgraph using a residual decomposition flow with signal independence regularization, and (3) a traffic prediction module that combines the predicted signals of each subgraph to generate final predictions. Extensive experiments on four traffic datasets validated the portability and effectiveness of TGDL. As a framework, TGDL can be flexibly applied to significantly improve a range of base models in spatiotemporal prediction tasks. Further analysis of the decomposed components confirmed that the proposed model adhered to the Component Independence Principle.

Limitation and Future Work: Our work has several limitations that warrant further investigation. First, while the current theoretical framework based on the CI principle establishes a sufficient condition for effective decomposition, it does not address necessity. Decompositions that do not satisfy the CI principle may still be beneficial in practice, but they are not covered by the current theoretical analysis. This leaves room for deeper theoretical exploration in future work. Second, we employ the CLUB estimator to approximate the mutual information between components in (36); however, it may introduce estimation errors, potentially affecting the accuracy of the regularization. Third, from a theoretical perspective, the CI Principle is not explicitly limited to traffic state data. However, in the present work, we restrict our attention to urban traffic data for both model design and empirical evaluation. Consequently, the applicability of the CI principle and the proposed TGDL framework to other spatiotemporal graph domains has not been explored in a systematic manner. Investigating such extensions remains beyond the scope of this study and is left for future work. Fourth, although our theoretical analysis characterizes how reducing data-induced uncertainty constrains the probability of prediction errors, it does not yield tight, metric-level guarantees for continuous regression losses such as MAE or RMSE. Establishing a stronger theoretical connection between uncertainty reduction and continuous prediction metrics remains an open direction for future research.

References

[1] J. Cheng et al., "POI-Enhancer: An LLM-based semantic enhancement framework for POI representation learning," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 11509–11517.

[2] J. Ji et al., "Precision CityShield against hazardous chemicals threats via location mining and self-supervised learning," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 3072–3080.

[3] K. H. Hettige et al., "AirPhyNet: Harnessing physics-guided neural networks for air quality prediction," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–12.

[4] C. Han et al., "Bridging traffic state and trajectory for dynamic road network and trajectory representation learning," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 11, pp. 11763–11771.

[5] Q. Zhang et al., "Efficient traffic prediction through spatio-temporal distillation," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 1093–1101.

[6] J. Wang et al., "High-resolution data on human behavior for effective COVID-19 policy-making—Wuhan City, Hubei Province, China, january 1-february 29, 2020," *China CDC Weekly*, vol. 5, no. 4, pp. 76–81, 2023.

[7] G. Jin et al., "Spatio-temporal graph neural networks for predictive learning in urban computing: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 10, pp. 5388–5408, Oct. 2024.

[8] J. Wang et al., "LibCity: An open library for traffic prediction," in *Proc. 29th Int. Conf. Adv. Geographic Inf. Syst.*, 2021, pp. 145–148.

[9] W. Zhang et al., "VecCity: A taxonomy-guided library for map entity representation learning," *Proc. IEEE Symp. Very Large-Scale Data Anal. Vis.*, vol. 18, no. 8, pp. 2575–2588, 2025.

[10] Y. Hong et al., "STKOpt: Automated spatio-temporal knowledge optimization for traffic prediction," in *Proc. Int. Conf. World Wide Web*, 2025, pp. 2238–2249.

[11] J. Ji et al., "STDEN: Towards physics-guided neural networks for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 4, pp. 4048–4056.

[12] J. Wang et al., "Empowering A* search algorithms with neural networks for personalized route recommendation," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 539–547.

[13] R. Dai et al., "Hybrid spatio-temporal graph convolutional network: Improving traffic prediction with navigation data," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3074–3082.

[14] J. Wang et al., "Traffic speed prediction and congestion source exploration: A deep learning method," in *Proc. Int. Conf. Data Mining*, 2016, pp. 499–508.

[15] J. Ji et al., "Spatio-temporal self-supervised learning for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 4356–4364.

[16] S. V. Kumar and L. Vanajakshi, "Short-term traffic flow prediction using seasonal arima model with limited input data," *Eur. Transport Res. Rev.*, vol. 7, no. 3, p. 21, 2015.

[17] I. Okutani and Y. J. Stephanedes, "Dynamic prediction of traffic volume through Kalman filtering theory," *Transp. Res. Part B, Methodological*, vol. 18, no. 1, pp. 1–11, 1984.

[18] D. Liu et al., "MSDR: Multi-step dependency relation networks for spatial temporal forecasting," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 1042–1050.

[19] Y. Li et al., "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–10.

[20] J. Wang et al., "Traffic flow prediction based on spatiotemporal potential energy fields," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 9073–9087, Sep. 2023.

[21] Z. Wu et al., "Graph wavenet for deep spatial-temporal graph modeling," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 1907–1913.

[22] J. Jiang et al., "PDFormer: Propagation delay-aware dynamic long-range transformer for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 4365–4373.

[23] Y. Yuan et al., "Unist: A prompt-empowered universal model for urban spatio-temporal prediction," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 4095–4106.

[24] Y. Fang et al., "Efficient large-scale traffic forecasting with transformers: A spatial data management perspective," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2025, pp. 307–317.

[25] J. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 1655–1661.

[26] X. Zhang et al., "Traffic flow forecasting with spatial-temporal graph diffusion network," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 17, pp. 15008–15015.

[27] J. Han et al., "BigST: Linear complexity spatio-temporal graph neural network for traffic forecasting on large-scale road networks," *Proc. VLDB Endowment*, vol. 17, no. 5, pp. 1081–1090, 2024.

[28] Y. Yao, X. Dai, and Y. Lv, "Leveraging heterogeneous experts with advantageous pattern memory learning for traffic prediction," in *Proc. IEEE Int. Conf. Data Eng.*, 2025, pp. 3342–3355.

[29] Z. Dong et al., "Heterogeneity-informed meta-parameter learning for spatiotemporal time series forecasting," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 631–641.

[30] Z. Fan, X. Song, and R. Shibasaki, "Cityspectrum: A non-negative tensor factorization approach," in *Proc. ACM Int. Joint Conf. Pervasive Ubiquitous Comput.*, 2014, pp. 213–223.

[31] J. Wang, J. Wu, Z. Wang, F. Gao, and Z. Xiong, "Understanding urban dynamics via context-aware tensor factorization with neighboring regularization," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 11, pp. 2269–2283, Nov. 2020.

[32] J. Deng et al., "Disentangling structured components: Towards adaptive, interpretable and scalable time series forecasting," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 8, pp. 3783–3800, Aug. 2024.

[33] G. Woo et al., "CoST: Contrastive learning of disentangled seasonal-trend representations for time series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–11.

[34] J. Wang et al., "Multilevel wavelet decomposition network for interpretable time series analysis," in *24th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2018, pp. 2437–2446.

[35] S. Soltani, "On the use of the wavelet decomposition for time series prediction," *Neurocomputing*, vol. 48, no. 1-4, pp. 267–277, 2002.

[36] Z. Shao et al., "Decoupled dynamic spatial-temporal graph neural network for traffic forecasting," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2733–2746, 2022.

[37] S. He, J. Ji, and M. Lei, "Decomposed spatio-temporal mamba for long-term traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 11772–11780.

[38] Z. Li et al., "SSL-STMFormer: Self-supervised learning spatio-temporal entanglement transformer for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 12130–12138.

[39] Y. Fang et al., "When spatio-temporal meet wavelets: Disentangled traffic forecasting via efficient spectral graph attention networks," in *Proc. IEEE Int. Conf. Data Eng.*, 2023, pp. 517–529.

[40] L. Cao et al., "Spatiotemporal-aware trend-seasonality decomposition network for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 11463–11471.

[41] C. Song et al., "Limits of predictability in human mobility," *Science*, vol. 327, no. 5968, pp. 1018–1021, 2010.

[42] J. Wang et al., "Predictability of road traffic and congestion in urban areas," *PLoS One*, vol. 10, no. 4, 2015, Art. no. e0121825.

[43] R. M. Fano and D. Hawkins, "Transmission of information: A statistical theory of communications," *Amer. J. Phys.*, vol. 29, no. 11, pp. 793–794, 1961.

[44] F. Chen et al., "Graph convolutional networks with Kalman filtering for traffic prediction," in *Proc. 28th Int. Conf. Adv. Geographic Inf. Syst.*, 2020, pp. 135–138.

[45] H. Yao et al., "Revisiting spatial-temporal similarity: A deep learning framework for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 5668–5675.

[46] J. Zhang, Y. Zheng, J. Sun, and D. Qi, "Flow prediction in spatio-temporal networks based on multitask deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 3, pp. 468–478, Mar. 2020.

[47] J. Ji et al., "Seeing the unseen: Learning basis confounder representations for robust traffic prediction," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2025, pp. 577–588.

[48] X. Yu et al., "BIGCity: A universal spatiotemporal model for unified trajectory and traffic state data analysis," in *Proc. IEEE Int. Conf. Data Eng.*, 2025, pp. 4455–4469.

[49] C.-C. M. Yeh et al., "RPMixer: Shaking up time series forecasting with random projections for large spatial-temporal data," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 3919–3930.

[50] Z. Li et al., "UrbanGPT: Spatio-temporal large language models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 5351–5362.

[51] P. Montero-Manso and R. J. Hyndman, "Principles and algorithms for forecasting groups of time series: Locality and globality," *Int. J. Forecasting*, vol. 37, no. 4, pp. 1632–1653, 2021.

[52] Z. Xu et al., "Ultra-flexible, explainable, and scalable traffic prediction with dynamic future routes," in *Proc. IEEE Int. Conf. Data Eng.*, 2025, pp. 3288–3301.

[53] J. Wang et al., "WHEN: A wavelet-DTW hybrid attention network for heterogeneous time series analysis," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 2361–2373.

[54] C. Yang et al., "Learning universal multi-level market irrationality factors to improve stock return forecasting," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2025, pp. 1739–1750.

[55] J. Wang et al., "Alphastock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks," in *Proc. 25th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2019, pp. 1900–1908.

[56] H. Ren, J. Wang, and W. X. Zhao, "Generative adversarial networks enhanced pre-training for insufficient electronic health records modeling," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 3810–3818.

[57] H. Ren, J. Wang, and W. X. Zhao, "RSD: A reinforced Siamese network with domain knowledge for early diagnosis," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1675–1684.

[58] C. Duchon and R. Hale, *Time Series Analysis in Meteorology and Climatology: An Introduction*, Hoboken, NJ, USA: Wiley, 2012.

[59] X. Qiu et al., "Empirical mode decomposition based ensemble deep learning for load demand time series forecasting," *Appl. Soft Comput.*, vol. 54, pp. 246–255, 2017.

[60] W. Zhou et al., "Real-time traffic flow forecasting via a novel method combining periodic-trend decomposition," *Sustainability*, vol. 12, no. 15, 2020, Art. no. 5891.

[61] K. Pholsena, L. Pan, and Z. Zheng, "Mode decomposition based deep learning model for multi-section traffic prediction," *World Wide Web*, vol. 23, no. 4, pp. 2513–2527, 2020.

[62] W. Weng et al., "A decomposition dynamic graph convolutional recurrent network for traffic forecasting," *Pattern Recognit.*, vol. 142, 2023, Art. no. 109670.

[63] G. Karypis and V. Kumar, "METIS: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices," Dept. Comput. Sci. Eng., Univ. Minnesota, pp. 21–33, 1997.
[64] C. Xie et al., "Distributed power-law graph computing: Theoretical and empirical analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 1673–1681.
[65] A. E. Feldmann and L. Foschini, "Balanced partitions of trees and applications," *Algorithmica*, vol. 71, no. 2, pp. 354–376, 2015.
[66] G. Karypis and V. Kumar, "Multilevel k-way hypergraph partitioning," in *Proc. 36th Annu. ACM/IEEE Des. Automat. Conf.*, 1999, pp. 343–348.
[67] A. Y. Ng, M. I. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Proc. Adv. Neural Inf. Process. Syst.*, 2001, pp. 849–856.
[68] T. Kawamoto, M. Tsubaki, and T. Obuchi, "Mean-field theory of graph neural networks in graph partitioning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4366–4376.
[69] M.-S. Chen, X.-R. Zhu, J.-Q. Lin, and C.-D. Wang, "Contrastive multiview attribute graph clustering with adaptive encoders," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 36, no. 4, pp. 7184–7195, Apr. 2025.
[70] Z. Tao et al., "Adversarial graph embedding for ensemble clustering," in *Proc. Int. Joint Conferences Artif. Intell. Org.*, 2019, pp. 3562–3568.
[71] T. Mallick et al., "Graph-partitioning-based diffusion convolutional recurrent neural network for large-scale traffic forecasting," *Transp. Res. Rec.*, vol. 2674, no. 9, pp. 473–488, 2020.
[72] S. Monaco, D. Apiletti, and G. Malnati, "Theory-guided deep learning algorithms: An experimental evaluation," *Electronics*, vol. 11, no. 18, 2022, Art. no. 2850.
[73] R. Wang et al., "Towards physics-informed deep learning for turbulent flow prediction," in *Proc. 26th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2020, pp. 1457–1466.
[74] M. Xie, Z. Jiang, and A. M. Sainju, "Geographical hidden Markov tree for flood extent mapping," in *Proc. 24th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2018, pp. 2545–2554.
[75] J. Ji et al., "Interpretable spatiotemporal deep learning model for traffic flow prediction based on potential energy fields," in *Proc. Int. Conf. Data Mining*, 2020, pp. 1076–1081.
[76] P. Deng et al., "Spatio-temporal neural structural causal models for bike flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 4242–4249.
[77] Y. Xia et al., "Deciphering spatio-temporal graph forecasting: A causal lens and treatment," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 37068–37088.
[78] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
[79] G. Boffetta et al., "Predictability: A way to characterize complexity," *Phys. Rep.*, vol. 356, no. 6, pp. 367–474, 2002.
[80] P. Domingos, "A unified bias-variance decomposition," in *Proc. Int. Conf. Mach. Learn.*, 2000, pp. 231–238.
[81] J. E. Taylor, G. A. Rousselet, and S. C. Sereno, "Can prediction error explain predictability effects on the n1 during picture-word verification?," *Imag. Neurosci.*, vol. 2, pp. 1–24, 2024.
[82] X. Geng et al., "Spatiotemporal multi-graph convolution network for ride-hailing demand forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 01, pp. 3656–3663.
[83] F. Li et al., "Automated spatio-temporal synchronous modeling with multiple graphs for traffic prediction," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1084–1093.
[84] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. Int. Joint Conf. Artif. Intell.*, 2018, pp. 3634–3640.
[85] P. Cheng et al., "CLUB: A contrastive log-ratio upper bound of mutual information," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1779–1788.
[86] S. Liu, E. Johns, and A. J. Davison, "End-to-end multi-task learning with attention," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 1871–1880.
[87] C. Chen et al., "Freeway performance measurement system: Mining loop detector data," *Transp. Res. Rec.*, vol. 1748, no. 1, pp. 96–102, 2001.
[88] S. Guo et al., "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 01, pp. 922–929.
[89] Z. Wu et al., "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proc. 26th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2020, pp. 753–763.
[90] W. Lu et al., "Traffic flow prediction for highway vehicle detectors through decomposition and machine learning," *Transp. Lett.*, vol. 17, no. 2, pp. 260–280, 2025.
[91] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, 2008.
[92] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.

Jiahao Ji received the BS degree in 2019 from Beihang University, where he is currently working toward the PhD degree with the School of Computer Science and Engineering. His research interests include spatiotemporal data mining, interpretable machine learning, and urban computing.

Jingyuan Wang received the PhD degree from the Department of Computer Science and Technology, Tsinghua University. He is currently a professor with the School of Computer Science and Engineering and School of Economics and Management, Beihang University. He is also the head of Beihang Interest Group on SmartCity (BIGSCity) and vice director of Beijing City Lab (BCL). His research interests data mining and machine learning, with special interests in smart cities and spatiotemporal data analytics. He was the recipient of the various national awards including NSFC Excellent Youth and first prize of technical invention of Ministry of Education.

Yu Mou received the BS degree in 2022 from Beihang University, where he currently working toward the graduation degree with the School of Computer Science and Engineering. His research interests include road representation learning and spatiotemporal data mining.

Cheng Long received the BEng degree from the South China University of Technology, China, in 2010, and the PhD degree from the Hong Kong University of Science and Technology, Hong Kong, in 2015. He is currently an associate professor with the College of Computing and Data Science, Nanyang Technological University. His research interests include data management and data mining.

Junjie Wu received the BE degree in civil engineering and the PhD degree in management science and engineering from Tsinghua University. He is currently a full professor with Information Systems Department, School of Economics and Management, Beihang University, director of the Social Computing Center, and the vice director of the Beijing Key Laboratory of Emergency Support Simulation Technologies for City Operations. His research interests include data mining and complex networks, with special interests in social media analytics and quantitative financial analytics. He was the recipient of various national awards including NSFC Excellent Youth, MOE Changjiang Young Scholars, and MOE Excellent Doctoral Dissertation.